Dataverse ELT project

SQL Tutor

Advanced SQL for the bronze-to-silver pipeline, and the theory behind it

- 25 lessons on DuckDB SQL
- Theory, a real-world case, popular usage and runnable examples in every lesson
- Every example shows its real result

Generated 2026-10-14 | results from DuckDB 1.5.6

CONTENTS

Lessons

Click a title to jump to it. The PDF viewer's bookmarks list every lesson and section.

Foundations

01 Grain, keys and normalization ..... foundation 6
02 Joins and cardinality ..... foundation 9
03 NULL semantics (three-valued logic) ..... foundation 13
04 Logical order of execution ..... foundation 16
05 Aggregation: GROUP BY, FILTER, ROLLUP ..... foundation to intermediate 19
06 Set operations: UNION, INTERSECT, EXCEPT ..... foundation 22

Query building blocks

07 CTEs, subqueries and recursion ..... intermediate 25
08 Window functions ..... intermediate 28
09 Latest-version dedup with ROW_NUMBER and QUALIFY ..... intermediate 31

Semi-structured data

10 Reading JSON files: schema-on-read and hive partitioning ..... intermediate 34
11 Nested types: STRUCT, LIST, MAP and JSON ..... intermediate 37
12 JSON functions: -> ->> JSONPath, json_type, from_json ..... intermediate 40
13 UNNEST: turning a list into rows ..... intermediate 43
14 EAV, pivot and unpivot ..... intermediate 47
15 Lambdas and list functions ..... intermediate 50

Types, time and reuse

16 Types, time zones and number pitfalls ..... intermediate 53
17 Macros: reusable SQL expressions ..... intermediate 56
18 Safe casting: CAST, TRY_CAST and validating raw values ..... intermediate 59

Pipeline engineering

19 Tables, views, temp tables and idempotent rebuilds ..... intermediate 62
20 Medallion layers and keeping history (SCD type 1 and 2) ..... advanced 65
21 Incremental loads and upserts (MERGE, ON CONFLICT) ..... advanced 69
22 Data quality as SQL: checks, severity and gates ..... advanced 72
23 Performance and storage: Parquet, pushdown and partitioning ..... advanced 75
24 Running SQL safely from Python: parameters, quoting, templates ..... intermediate 78
25 Dialect portability: DuckDB versus PostgreSQL (and the SQL standard) ..... advanced 81

ABOUT

About this module

A self-study module with 25 lessons on the SQL used in this project (DuckDB dialect) and the theory around it. Every lesson has the same four parts:

1. **Theory**: the base concepts and rules.
2. **Real-world case**: a problem from THIS project (or one it will meet next) that needed the technique.
3. **Popular usage**: where the technique shows up in industry.
4. **Example code**: runnable SQL / Python, executed live against a small sample shaped like your Dataverse records.

Quick start

Run from the project folder, with the project's virtual environment (DuckDB is already installed there).

List the lessons:

Terminal
```
.venv/Scripts/python sql_tutor/tutor.py
```

Read and run one lesson, by number or by part of its name (the code is executed and its result printed under each block):

Terminal
```
.venv/Scripts/python sql_tutor/tutor.py 9
```

Terminal
```
.venv/Scripts/python sql_tutor/tutor.py dedup
```

Read the text only, without running the code:

Terminal
```
.venv/Scripts/python sql_tutor/tutor.py 9 --no-run
```

Check that every example in every lesson still runs (useful after editing a lesson or the project's SQL):

Terminal
```
.venv/Scripts/python sql_tutor/tutor.py --check
```

You can also just open `lessons/*.md` in your editor; they are plain Markdown.

Suggested path

1. **Foundations**: 01, 02, 03, 04, 05, 06. Everything else builds on these.
2. **Reshaping**: 07, 08, 09, then 10 to 15 (the semi-structured JSON toolbox), then 16 to 18 (types and casting).
3. **Engineering**: 19, 20, 21, 22 (how pipelines stay correct), 23 (how they stay fast), 24, 25 (safety and portability).

The lessons are independent: each runs in a fresh in-memory database, so you can jump to any of them.

Map from the topic list to lessons

Topic	Lesson(s)
CTAS and temp tables	19
Schema-on-read, hive partitioning	10
STRUCT / LIST / JSON types	11
UNNEST and ORDINALITY	13
JSON extraction	12
EAV and pivot with FILTER	14, 05
CTEs and recursion	07
Lambdas and list functions	15
Macros, time-zone handling	17, 16
Safe casting	18
Window functions, QUALIFY dedup	08, 09
Joining on the full version key	02
Anti join, HAVING, UNION ALL checks	02, 05, 06, 22
SQL from Python	24
Grain, keys, normalization	01
NULL semantics	03
Order of execution	04
Medallion, SCD, idempotency, incremental	20, 19, 21
Data quality	22
Performance and storage	23
Portability and PostgreSQL	25

The sample data

`sample_data/` holds two extraction runs of one dataverse (`orpertanian`) in the same folder layout as the raw bucket (`extract_date=.../run_id=.../entity=dataset_metadata/dataverse=.../part-00000.jsonl`). Four datasets are chosen so each lesson has something to show:

Dataset	What is special about it
DS1	in both runs; title and file list changed in run B (dedup, history)
DS2	deaccessioned: no `latestVersion` (NULL handling, the UNNEST gotcha)
DS3	new in run B; invalid publication date `2025-13-45` ; timestamp with a `+07:00` offset (safe casting, time zones)
DS4	a DRAFT, only in run A (what disappeared between runs)

`sample_data.py` regenerates the files if you delete the folder.

How the code blocks run

Only the blocks under "## 4. Example code" are executed:

- ```sql blocks run in DuckDB; the result of the last statement is printed.
- ```python blocks run with these names available: `con` (the DuckDB connection), `duckdb`, `run_sql_file(name)` (runs a helper such as `setup_bronze.sql`), `build_silver()` (runs the project's real `flatten_datasets_metadata.sql` on the sample files) and `HERE` (this folder).
- Any other fence (```postgres, ```text) is shown but never run.

Helper files: `setup_bronze.sql` builds `bronze_meta`; `setup_fields.sql` builds `stg_fields` (needs `bronze_meta` first).

Notes

- Lessons 20 and 22 run the project's actual `flatten_datasets_metadata.sql`. If you change that file, run `--check`: a failure there means the lesson text needs updating (or the SQL change broke something).
- Lesson 23 writes about 55 MB of temporary files (CSV, JSON, Parquet) and removes them when it finishes.
- All SQL targets the DuckDB version in your venv. Lesson 25 lists what changes for PostgreSQL.

LESSON 01

Grain, keys and normalization

Level: foundation | In this project: the four child tables `silver_dataset_authors / _subjects / _keywords / _grants` | Needs: nothing

1. Theory

Grain is the answer to "what does ONE row of this table represent?". Say it in a sentence before you write any query: "one row per dataset", "one row per (dataset, author)". Most wrong numbers in reporting (inflated sums, duplicated rows) are grain mistakes.

Keys identify rows:

- Natural (business) key: a value that means something in the real world. Here: the DOI, `global_id`.
- Surrogate key: an id invented by a system. Here: Dataverse's numeric `dataset_id`, or an auto-increment.
- Composite key: several columns together, e.g. (`global_id`, `author_position`).
- A PRIMARY KEY is unique and never NULL. A FOREIGN KEY is a column that points at a key in another table.

First normal form (1NF): every cell holds ONE value. A cell holding a list ("Doe, Jane; Roe, Rick") breaks 1NF. The fix is a child table: one row per item, plus the parent's key. Second and third normal form continue the idea: every column should depend on the whole key and nothing but the key, so a fact (an author's affiliation, say) is stored once instead of copied into many rows.

Trade-off: normalized tables avoid duplicated facts and make filters exact. Wide, denormalized tables need fewer joins and are faster to read. A common design keeps silver normalized and builds wide tables for dashboards (gold) on top.

2. Real-world case

The Dataverse API returns authors as a list inside each dataset. Imagine the first version of the silver table stored them as one text column, `'Doe, Jane; Roe, Rick'`. Then:

- "Which datasets did Ana write?" needs `LIKE '%Ana%'`, which also matches `Hanafi`.
- "How many authors per dataset?" needs string splitting.
- "Authors with an ORCID" has nowhere to put the ORCID.

The project splits every multi-valued field into its own table whose key states the grain: (`global_id`, `author_position`), (`global_id`, `subject_position`), and so on.

3. Popular usage

- Orders and order lines; articles and tags; students and courses (a many-to-many needs a "bridge" table).
- Every warehouse fact table starts with a written grain statement ("one row per order line per day").
- dbt tests `unique` and `not_null` on the key columns are grain tests.

4. Example code

SQL

```sql
-- The "list in a cell" design: authors crammed into one text column.
CREATE TABLE datasets_flat AS
SELECT * FROM (VALUES
    ('DS1', 'Rice yield survey',    'Doe, Jane; Roe, Rick'),
    ('DS3', 'Fish landing records', 'Kim, Ana'),
    ('DS5', 'Market price series',  'Hanafi, Budi')
) AS t(global_id, title, authors);
```

SQL

```sql
-- "Which datasets did Ana write?"  ILIKE also matches the "ana" inside Hanafi: a wrong answer.
SELECT global_id, authors FROM datasets_flat WHERE authors ILIKE '%ana%' ORDER BY global_id;
```

Result

```
global_id | authors
----------+-------------
DS3       | Kim, Ana
DS5       | Hanafi, Budi
(2 rows)
```

SQL

```sql
-- Normalized: one row per (dataset, author). The primary key states the grain.
CREATE TABLE dataset_authors (
    global_id       VARCHAR NOT NULL,
    author_position INTEGER NOT NULL,
    author_name     VARCHAR NOT NULL,
    PRIMARY KEY (global_id, author_position)
);
INSERT INTO dataset_authors VALUES
    ('DS1', 1, 'Doe, Jane'), ('DS1', 2, 'Roe, Rick'), ('DS3', 1, 'Kim, Ana'), ('DS5', 1, 'Hanafi, Budi');
```

SQL

```sql
-- Exact match now: only the real Ana.
SELECT global_id FROM dataset_authors WHERE author_name = 'Kim, Ana';
```

Result

```
global_id
---------
DS3
(1 row)
```

SQL

```sql
-- Authors per dataset is a plain GROUP BY.
SELECT global_id, count(*) AS n_authors FROM dataset_authors GROUP BY global_id ORDER BY global_id;
```

Result

```
global_id | n_authors
----------+----------
DS1       | 2
DS3       | 1
DS5       | 1
(3 rows)
```

SQL

```sql
-- A grain test: this must return 0 rows. (dbt calls it a "unique" test.)
SELECT global_id, author_position, count(*) AS n
FROM dataset_authors
GROUP BY ALL
HAVING count(*) > 1;
```

Result

```
global_id | author_position | n
----------+-----------------+--
(0 rows)
```

Python

```python
# The primary key enforces the grain: a second row for the same (dataset, position) is rejected.
try:
    con.execute("INSERT INTO dataset_authors VALUES ('DS1', 1, 'Someone Else')")
except duckdb.ConstraintException as e:
    print("Rejected:", str(e).splitlines()[0])
```

Result

```
Rejected: Constraint Error: Duplicate key "global_id: DS1, author_position: 1" violates primary key constraint.
```

LESSON 02

Joins and cardinality

Level: foundation | In this project: child tables joined to `silver_datasets`, `LEFT JOIN pivoted`, `ANTI JOIN` checks | Needs: lesson 01

1. Theory

A join combines rows of two tables whose key columns match.

Join	Keeps	Typical use
INNER	rows with a match on both sides	attach details to rows that must have them
LEFT	every left row, NULLs where there is no match	"keep all datasets even without authors"
RIGHT / FULL	every right row / every row of both	reconciling two lists
CROSS	every left row paired with every right row	calendars, grids, combinations
SEMI (`EXISTS`)	left rows that have a match, each once	"datasets that have at least one author"
ANTI (`NOT EXISTS`)	left rows that have NO match	orphans, missing data

`ON a.k = b.k` states the condition. `USING (k)` is shorthand when both columns have the same name and keeps only one copy of `k` in `SELECT *`.

Cardinality is how many rows match on each side: 1:1, 1:N, N:N. Joining to the "many" side multiplies the "one" side's rows. This is called **fan-out**. If the join key is not unique on a side, rows multiply silently and every `sum()` and `count(*)` after the join is inflated.

Before any join ask: "is my key unique on each side?" If not, add the missing column to the key.

2. Real-world case

`stg_fields` holds the fields of EVERY run, so dataset DS1 appears once per run. `silver_datasets` keeps one version per dataset (the newest run). The first draft of the author table joined on `global_id` only: each author appeared once per run (the old version AND the kept version). The fix is to join on the whole version key, `global_id` AND `source_run_id`. It is the `ON` clause in every child table of `flatten_datasets_metadata.sql`.

3. Popular usage

- LEFT JOIN from a "calendar spine" (CROSS JOIN of all months and all categories) so a report shows zeros instead of missing rows.
- Anti join for data quality: orphans, missing dimension members, "customers without orders".
- Semi join (`EXISTS`) to filter without multiplying rows.

4. Example code

SQL

```sql
-- Which version of each dataset was kept (run B), and every author row of every run.
CREATE TABLE kept_versions AS
SELECT * FROM (VALUES ('DS1', 'B'), ('DS3', 'B')) AS t(global_id, source_run_id);

CREATE TABLE author_rows AS
SELECT * FROM (VALUES
    ('DS1', 'A', 'Doe, Jane'), ('DS1', 'A', 'Roe, Rick'),   -- old version of DS1
    ('DS1', 'B', 'Doe, Jane'), ('DS1', 'B', 'Roe, Rick'),   -- kept version of DS1
    ('DS3', 'B', 'Kim, Ana')
) AS t(global_id, run_id, author_name);
```

SQL

```sql
-- WRONG: join on global_id only. DS1 has 2 authors but shows 4 rows (fan-out).
SELECT k.global_id, count(*) AS author_rows
FROM kept_versions k
JOIN author_rows a ON a.global_id = k.global_id
GROUP BY k.global_id ORDER BY 1;
```

Result

```
global_id | author_rows
----------+------------
DS1       | 4
DS3       | 1
(2 rows)
```

SQL

```sql
-- RIGHT: join on the full version key. DS1 -> 2 authors.
SELECT k.global_id, count(*) AS author_rows
FROM kept_versions k
JOIN author_rows a ON a.global_id = k.global_id AND a.run_id = k.source_run_id
GROUP BY k.global_id ORDER BY 1;
```

Result

```
global_id | author_rows
----------+------------
DS1       | 2
DS3       | 1
(2 rows)
```

SQL

```sql
CREATE TABLE ds AS SELECT * FROM (VALUES ('DS1'), ('DS2'), ('DS3')) AS t(global_id);
```

SQL

```sql
-- LEFT JOIN keeps DS2 even though it has no authors (its author_name is NULL).
SELECT d.global_id, a.author_name
FROM ds d
LEFT JOIN author_rows a ON a.global_id = d.global_id AND a.run_id = 'B'
ORDER BY 1, 2;
```

Result

```
global_id | author_name
----------+------------
DS1       | Doe, Jane
DS1       | Roe, Rick
DS2       | NULL
DS3       | Kim, Ana
(4 rows)
```

SQL

```sql
-- ANTI JOIN (DuckDB syntax): datasets with no author row at all. Portable form: NOT EXISTS.
SELECT global_id FROM ds ANTI JOIN author_rows USING (global_id);
```

Result

```
global_id
---------
DS2
(1 row)
```

SQL

```sql
-- SEMI JOIN via EXISTS: each dataset once, even though DS1 matches 4 author rows.
SELECT global_id FROM ds d
WHERE EXISTS (SELECT 1 FROM author_rows a WHERE a.global_id = d.global_id)
ORDER BY 1;
```

Result

```
global_id
---------
DS1
DS3
(2 rows)
```

SQL

```sql
-- CROSS JOIN as a calendar spine: a zero-filled "files uploaded per dataverse per month" report.
CREATE TABLE months AS SELECT unnest(['2025-01', '2025-02', '2025-03']) AS month;
CREATE TABLE uploads AS
SELECT * FROM (VALUES ('orpertanian', '2025-01', 5), ('orpertanian', '2025-03', 2), ('perikanan', '2025-02', 7))
       AS t(dataverse, month, n_files);
```

SQL

```sql
SELECT dv.dataverse, m.month, COALESCE(u.n_files, 0) AS n_files
FROM (SELECT DISTINCT dataverse FROM uploads) dv
CROSS JOIN months m
LEFT JOIN uploads u ON u.dataverse = dv.dataverse AND u.month = m.month
ORDER BY 1, 2;
```

Result

```
dataverse   | month   | n_files
------------+---------+--------
orpertanian | 2025-01 | 5
orpertanian | 2025-02 | 0
orpertanian | 2025-03 | 2
perikanan   | 2025-01 | 0
perikanan   | 2025-02 | 7
perikanan   | 2025-03 | 0
(6 rows)
```

LESSON 03

NULL semantics (three-valued logic)

Level: foundation | In this project: `has_custom_terms`, `dataset without a title` check, `NULLS LAST`, deaccessioned datasets | Needs: nothing

1. Theory

NULL means "unknown / missing", not zero and not empty text. Comparing anything with an unknown gives an unknown, so SQL logic has THREE values: TRUE, FALSE and NULL (unknown).

- `x = NULL` and `x <> NULL` are never TRUE. Use `x IS NULL` / `x IS NOT NULL`.
- `WHERE` keeps only rows where the condition is TRUE. FALSE and NULL rows are both dropped.
- `NULL AND FALSE` is FALSE, `NULL AND TRUE` is NULL, `NULL OR TRUE` is TRUE, `NOT NULL` is NULL.
- Aggregates skip NULLs: `count(col)` counts non-NULL values, `count(*)` counts rows, `sum` and `avg` ignore NULLs.
- `NULL` in the list of `x NOT IN (...)` makes the whole condition NULL: no rows come back. Use `NOT EXISTS`.
- `IS DISTINCT FROM` / `IS NOT DISTINCT FROM` compare like `<>` / `=` but treat two NULLs as equal.
- Helpers: `COALESCE(a, b, ...)` first non-NULL; `NULLIF(a, b)` NULL if a = b; `ORDER BY ... NULLS FIRST|LAST`.

2. Real-world case

The Dataverse field `termsOfUse` is NULL for some datasets, `''` for some, and spaces for others. The silver flag `has_custom_terms` must be TRUE only when real text exists. The naive `terms <> ''` silently drops the NULL rows and wrongly keeps the whitespace ones. The project's version is `COALESCE(trim(terms_of_use_raw), '') <> ''`.

The same care shows in the quality checks: "dataset without a title" must skip deaccessioned datasets, whose title is legitimately NULL.

3. Popular usage

- Cleaning: `NULLIF(trim(col), '')` turns blank strings into NULL so later logic needs only `IS NULL`.
- Safe division: `x / NULLIF(y, 0)` returns NULL instead of an error.
- Change detection between two snapshots: `old IS DISTINCT FROM new`.
- Defaults in reports: `COALESCE(sum(x), 0)`.

4. Example code

SQL

```sql
CREATE TABLE terms AS
SELECT * FROM (VALUES
    ('DS1', ''),
    ('DS2', NULL),
    ('DS3', 'Cite the source.'),
    ('DS4', '   ')
) AS t(global_id, terms_of_use);
```

SQL

```sql
-- Naive: NULL (DS2) vanishes silently, and the whitespace-only DS4 is wrongly kept.
SELECT global_id FROM terms WHERE terms_of_use <> '' ORDER BY 1;
```

Result

```
global_id
---------
DS3
DS4
(2 rows)
```

SQL

```sql
-- Correct: only DS3 has real terms.
SELECT global_id FROM terms WHERE COALESCE(trim(terms_of_use), '') <> '' ORDER BY 1;
```

Result

```
global_id
---------
DS3
(1 row)
```

SQL

```sql
-- The same idea with NULLIF: blank text becomes NULL, then one IS NOT NULL test is enough.
SELECT global_id, NULLIF(trim(terms_of_use), '') AS cleaned FROM terms ORDER BY 1;
```

Result

```
global_id | cleaned
----------+-----------------
DS1       | NULL
DS2       | NULL
DS3       | Cite the source.
DS4       | NULL
(4 rows)
```

SQL

```sql
-- count(*) counts rows, count(col) skips NULLs, count(DISTINCT col) counts distinct non-NULL values.
SELECT count(*) AS all_rows, count(terms_of_use) AS non_null, count(DISTINCT terms_of_use) AS distinct_values FROM terms;
```

Result

```
all_rows | non_null | distinct_values
---------+----------+----------------
4        | 3        | 3
(1 row)
```

SQL

```sql
-- Three-valued logic in one row (a-e are NULL, FALSE, TRUE, NULL, TRUE).
SELECT
    NULL AND TRUE              AS a,
    NULL AND FALSE             AS b,
    NULL OR TRUE               AS c,
    NULL = NULL                AS d,
    NULL IS NOT DISTINCT FROM NULL AS e;
```

Result

```
a    | b     | c    | d    | e
-----+-------+------+------+-----
NULL | false | true | NULL | true
(1 row)
```

SQL

```sql
CREATE TABLE blocked AS SELECT * FROM (VALUES ('DS2'), (NULL)) AS t(global_id);
```

SQL

```sql
-- NOT IN trap: one NULL in the subquery makes the answer empty.
SELECT global_id FROM terms WHERE global_id NOT IN (SELECT global_id FROM blocked);
```

Result

```
global_id
---------
(0 rows)
```

SQL

```sql
-- NOT EXISTS gives the answer you wanted: everything except DS2.
SELECT global_id FROM terms t
WHERE NOT EXISTS (SELECT 1 FROM blocked b WHERE b.global_id = t.global_id)
ORDER BY 1;
```

Result

```
global_id
---------
DS1
DS3
DS4
(3 rows)
```

SQL

```sql
-- Change detection: <> misses DS3 (NULL -> 'X'); IS DISTINCT FROM catches it and ignores DS2 (NULL -> NULL).
SELECT id, old_title, new_title,
       old_title <> new_title            AS changed_naive,
       old_title IS DISTINCT FROM new_title AS changed_correct
FROM (VALUES ('DS1', 'A', 'B'), ('DS2', NULL, NULL), ('DS3', NULL, 'X')) AS t(id, old_title, new_title)
ORDER BY id;
```

Result

```
id  | old_title | new_title | changed_naive | changed_correct
----+-----------+-----------+---------------+----------------
DS1 | A         | B         | true          | true
DS2 | NULL      | NULL      | NULL          | false
DS3 | NULL      | X         | NULL          | true
(3 rows)
```

LESSON 04

Logical order of execution

Level: foundation | In this project: why the dedup uses `QUALIFY`, why checks use `HAVING` | Needs: nothing

1. Theory

You WRITE `SELECT ... FROM ... WHERE ...`, but the database THINKS in this order:

```
FROM / JOIN  ->  WHERE  ->  GROUP BY  ->  aggregates  ->  HAVING  ->  window functions
             ->  QUALIFY  ->  SELECT (expressions, aliases)  ->  DISTINCT  ->  ORDER BY  ->  LIMIT
```

Consequences you meet daily:

- `WHERE` runs before grouping, so it cannot contain aggregates. Filter groups with `HAVING`.
- `WHERE` runs before window functions, so it cannot filter on them. Use `QUALIFY` (DuckDB, Snowflake, BigQuery) or wrap the query in a subquery / CTE and filter outside.
- `WHERE` filters ROWS before they are counted, `HAVING` filters GROUPS after. Same condition, different meaning.
- Standard SQL does not let `WHERE` use a `SELECT` alias (it does not exist yet). DuckDB is lenient and allows it, but code that relies on it will not port to Postgres.
- `ORDER BY` runs last, so it can use aliases, and `LIMIT` applies after sorting.

2. Real-world case

Three questions about the dataset catalogue, one table, three different clauses:

1. "Which datasets appear in at least two runs?" is a condition on a group, so `HAVING count(*) >= 2`.
2. "How many versions had files?" filters rows first, so `WHERE n_files > 0`, then counts.
3. "Newest version of each dataset" filters on a window function, so `QUALIFY`. This is the dedup in `silver_datasets`.

Putting a condition in the wrong clause either fails with an error (good) or quietly returns a different number (bad).

3. Popular usage

- Interview classic: explain why `WHERE count(*) > 1` fails.
- "Top N per group" in every analytics stack: window function in a CTE or `QUALIFY`.
- Reading query plans and optimizer behaviour starts from this logical order.

4. Example code

SQL

```sql
CREATE TABLE versions AS
SELECT * FROM (VALUES
    ('DS1', 'A', 2, '2025-01-01'),
    ('DS1', 'B', 3, '2025-06-01'),
    ('DS2', 'A', 0, NULL),
    ('DS3', 'B', 1, '2025-05-20'),
    ('DS3', 'A', 0, '2025-05-01')
) AS t(global_id, run_id, n_files, updated);
```

SQL

```sql
-- HAVING filters groups: datasets that appear in 2 or more runs.
SELECT global_id, count(*) AS n_runs FROM versions GROUP BY global_id HAVING count(*) >= 2 ORDER BY 1;
```

Result

```
global_id | n_runs
----------+-------
DS1       | 2
DS3       | 2
(2 rows)
```

SQL

```sql
-- WHERE filters rows BEFORE they are counted: only versions that had files. Compare DS3: 1, not 2.
SELECT global_id, count(*) AS versions_with_files FROM versions WHERE n_files > 0 GROUP BY global_id ORDER BY 1;
```

Result

```
global_id | versions_with_files
----------+--------------------
DS1       | 2
DS3       | 1
(2 rows)
```

SQL

```sql
-- QUALIFY filters on a window function: the newest version per dataset.
SELECT global_id, run_id, updated
FROM versions
QUALIFY row_number() OVER (PARTITION BY global_id ORDER BY run_id DESC) = 1
ORDER BY global_id;
```

Result

```
global_id | run_id | updated
----------+--------+-----------
DS1       | B      | 2025-06-01
DS2       | A      | NULL
DS3       | B      | 2025-05-20
(3 rows)
```

Python

```python
# The wrong placements fail loudly. Read the messages: they tell you which clause is wrong.
bad_queries = [
    "SELECT global_id FROM versions WHERE count(*) > 1 GROUP BY global_id",
    "SELECT global_id FROM versions WHERE row_number() OVER (PARTITION BY global_id ORDER BY run_id) = 1",
]
for sql in bad_queries:
    try:
        con.execute(sql)
    except duckdb.Error as e:
        print(str(e).splitlines()[0])
```

Result

```
Binder Error: WHERE clause cannot contain aggregates!
Binder Error: WHERE clause cannot contain window functions!
```

SQL

```sql
-- The portable way to filter on a window function: compute it in a subquery, filter outside.
SELECT global_id, run_id, updated
FROM (SELECT *, row_number() OVER (PARTITION BY global_id ORDER BY run_id DESC) AS rn FROM versions)
WHERE rn = 1
ORDER BY global_id;
```

Result

```
global_id | run_id | updated
----------+--------+-----------
DS1       | B      | 2025-06-01
DS2       | A      | NULL
DS3       | B      | 2025-05-20
(3 rows)
```

LESSON 05

Aggregation: GROUP BY, FILTER, ROLLUP

Level: foundation to intermediate | In this project: `max(...) FILTER (WHERE ...)`, `count(DISTINCT ...)`, `HAVING count(*) > 1` | Needs: lesson 04

1. Theory

An aggregate collapses many rows into one value per group: `count`, `sum`, `avg`, `min`, `max`, `string_agg`, `list`. `GROUP BY` defines the groups. Every column in `SELECT` must be either in `GROUP BY` or inside an aggregate (DuckDB offers `GROUP BY ALL` to group by every non-aggregate column).

Useful details:

- Aggregates ignore NULLs (lesson 03). `count(*)` counts rows.
- `count(DISTINCT x)` counts different values. It is slower than `count(x)`.
- **Conditional aggregation**: `agg(...) FILTER (WHERE cond)` (standard SQL) or `agg(CASE WHEN cond THEN x END)` (works everywhere). It computes several differently-filtered numbers in ONE pass.
- **Ordered aggregates**: `string_agg(x, ', ' ORDER BY x)`, `list(x ORDER BY y)`.
- **Subtotals**: `GROUP BY ROLLUP (a, b)` adds subtotal rows (by a, then grand total). `GROUPING SETS` and `CUBE` choose other combinations. `GROUPING(col)` is 1 on rows where `col` was rolled up.
- **Safe ratios**: divide by `NULLIF(denominator, 0)`.

2. Real-world case

A data steward asks for a monthly report: per dataverse, how many datasets, how many files, how many files are restricted, what share that is, with a total line. One query, no loops: conditional counts for each restriction class, `NULLIF` so an empty dataverse does not divide by zero, `ROLLUP` for the totals. The project uses the same building blocks: `FILTER` pivots fields into columns (lesson 14), `HAVING count(*) > 1` finds duplicate keys (lesson 22).

3. Popular usage

- KPI dashboards: one row per day or category with several conditional counts.
- Funnel reports: `count(*) FILTER (WHERE step >= 2) / count(*)`.
- Finance-style reports with subtotals and grand totals (`ROLLUP`).
- `string_agg` / `list` to build comma-separated tag lists or arrays per entity.

4. Example code

SQL

```sql
CREATE TABLE dataset_summary AS
SELECT * FROM (VALUES
    ('DS1', 'orpertanian', 3, 1),
    ('DS2', 'orpertanian', 0, 0),
    ('DS3', 'orpertanian', 1, 0),
    ('DS6', 'perikanan',  10, 0),
    ('DS7', 'perikanan',   4, 4)
) AS t(global_id, dataverse, n_files, n_restricted);
```

SQL

```sql
-- Basic GROUP BY with several aggregates.
SELECT dataverse, count(*) AS n_datasets, sum(n_files) AS n_files, sum(n_restricted) AS n_restricted
FROM dataset_summary GROUP BY dataverse ORDER BY dataverse;
```

Result

```
dataverse   | n_datasets | n_files | n_restricted
------------+------------+---------+-------------
orpertanian | 3          | 4       | 1
perikanan   | 2          | 14      | 4
(2 rows)
```

SQL

```sql
-- Conditional aggregation with FILTER, and the portable CASE form next to it (same numbers).
SELECT dataverse,
       count(*) FILTER (WHERE n_restricted > 0)                    AS datasets_with_restricted,
       sum(CASE WHEN n_restricted > 0 THEN 1 ELSE 0 END)           AS same_with_case,
       count(*) FILTER (WHERE n_files = 0)                         AS datasets_without_files
FROM dataset_summary GROUP BY dataverse ORDER BY dataverse;
```

Result

```
dataverse   | datasets_with_restricted | same_with_case | datasets_without_files
------------+--------------------------+----------------+-----------------------
orpertanian | 1                        | 1              | 1
perikanan   | 1                        | 1              | 0
(2 rows)
```

SQL

```sql
-- A ratio that survives an empty group: NULLIF turns a zero denominator into NULL.
SELECT dataverse, round(100.0 * sum(n_restricted) / NULLIF(sum(n_files), 0), 1) AS pct_restricted
FROM dataset_summary GROUP BY dataverse ORDER BY dataverse;
```

Result

```
dataverse   | pct_restricted
------------+---------------
orpertanian | 25.0
perikanan   | 28.6
(2 rows)
```

SQL

```sql
-- HAVING keeps groups: only dataverses with 3 or more datasets (WHERE could not do this, lesson 04).
SELECT dataverse, count(*) AS n_datasets
FROM dataset_summary GROUP BY dataverse HAVING count(*) >= 3 ORDER BY dataverse;
```

Result

```
dataverse   | n_datasets
------------+-----------
orpertanian | 3
(1 row)
```

SQL

```sql
-- ROLLUP adds a subtotal per dataverse and a grand total. GROUPING() tells total rows from data rows.
SELECT CASE WHEN grouping(dataverse) = 1 THEN '(all)' ELSE dataverse END AS dataverse,
       CASE WHEN grouping(dataverse) = 1 THEN '(grand total)'
            WHEN grouping(global_id) = 1 THEN '(subtotal)'
            ELSE global_id END AS dataset,
       sum(n_files) AS n_files
FROM dataset_summary
GROUP BY ROLLUP (dataverse, global_id)
ORDER BY grouping(dataverse), dataverse, grouping(global_id), dataset;
```

Result

```
dataverse   | dataset       | n_files
------------+---------------+--------
orpertanian | DS1           | 3
orpertanian | DS2           | 0
orpertanian | DS3           | 1
orpertanian | (subtotal)    | 4
perikanan   | DS6           | 10
perikanan   | DS7           | 4
perikanan   | (subtotal)    | 14
(all)       | (grand total) | 18
(8 rows)
```

SQL

```sql
-- Ordered aggregates: a readable list of dataset ids per dataverse, and the same as a real list value.
SELECT dataverse,
       string_agg(global_id, ', ' ORDER BY global_id)       AS ids_text,
       list(global_id ORDER BY n_files DESC)                AS ids_by_size
FROM dataset_summary GROUP BY dataverse ORDER BY dataverse;
```

Result

```
dataverse   | ids_text      | ids_by_size
------------+---------------+----------------
orpertanian | DS1, DS2, DS3 | [DS1, DS3, DS2]
perikanan   | DS6, DS7      | [DS6, DS7]
(2 rows)
```

LESSON 06

Set operations: UNION, INTERSECT, EXCEPT

Level: foundation | In this project: `qa_checks` is eight SELECTs joined by `UNION ALL` | Needs: nothing

1. Theory

Set operations combine the ROWS of two queries that have the same number of columns (matched by position, with compatible types):

Operator	Result
`UNION ALL`	all rows of both, duplicates kept (cheap: no sorting or hashing)
`UNION`	all rows of both, duplicates removed
`INTERSECT`	rows that appear in both
`EXCEPT`	rows of the first that are NOT in the second (order matters)

Joins combine COLUMNS side by side. Set operations stack ROWS on top of each other.

Rules of thumb:

- Default to `UNION ALL`. Use `UNION` only when you really want de-duplication, because it hides repeated rows and costs more.
- `INTERSECT` and `EXCEPT` treat two NULLs as equal (unlike `=`).
- DuckDB adds `UNION ALL BY NAME`: columns are matched by name, missing ones become NULL. It is handy when files have drifted schemas.

2. Real-world case

1. **Many checks, one table.** `qa_checks` has the same three columns for every check, so each check is a SELECT and `UNION ALL` stacks them. `UNION` would be wrong: two checks with the same name, severity and count would be merged into one.
2. **What changed between two extraction runs?** Datasets in run A that are missing from run B (deleted or hidden) are `A EXCEPT B`. Datasets new in B are `B EXCEPT A`. Datasets in both are `A INTERSECT B`.

3. Popular usage

- "Stack monthly tables / yearly partitions into one" with `UNION ALL`.
- Reconciling two systems: `EXCEPT` both ways until both sides are empty.
- Combining files whose columns drifted over time (`UNION ALL BY NAME`).

4. Example code

Python

```
run_sql_file("setup_bronze.sql")   # bronze_meta: 3 datasets in run A, 3 in run B
```

SQL

```sql
-- In run A but not in run B: what disappeared between the runs?
WITH a AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%aaaaaa'),
     b AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%bbbbbb')
SELECT id FROM a EXCEPT SELECT id FROM b;
```

Result

```
id
---
DS4
(1 row)
```

SQL

```sql
-- In run B but not in run A: what is new?
WITH a AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%aaaaaa'),
     b AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%bbbbbb')
SELECT id FROM b EXCEPT SELECT id FROM a;
```

Result

```
id
---
DS3
(1 row)
```

SQL

```sql
-- In both runs.
WITH a AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%aaaaaa'),
     b AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%bbbbbb')
SELECT id FROM a INTERSECT SELECT id FROM b ORDER BY id;
```

Result

```
id
---
DS1
DS2
(2 rows)
```

SQL

```sql
-- UNION ALL keeps every row (6); UNION removes duplicates (4 distinct datasets).
WITH a AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%aaaaaa'),
     b AS (SELECT data.identifier AS id FROM bronze_meta WHERE run_id LIKE '%bbbbbb')
SELECT (SELECT count(*) FROM (SELECT id FROM a UNION ALL SELECT id FROM b)) AS union_all_rows,
       (SELECT count(*) FROM (SELECT id FROM a UNION     SELECT id FROM b)) AS union_rows;
```

Result

```
union_all_rows | union_rows
---------------+-----------
6              | 4
(1 row)
```

SQL

```sql
-- The qa_checks pattern: each check is one SELECT with the same columns; UNION ALL stacks them.
SELECT 'released dataset without citation fields' AS check_name, 'error' AS severity, count(*) AS bad_rows
FROM bronze_meta WHERE data.latestVersion IS NOT NULL AND len(data.latestVersion.metadataBlocks.citation.fields) = 0
UNION ALL
SELECT 'deaccessioned dataset', 'warn', count(*) FROM bronze_meta WHERE data.latestVersion IS NULL
UNION ALL
SELECT 'draft version', 'warn', count(*) FROM bronze_meta WHERE data.latestVersion.versionState = 'DRAFT';
```

Result

```
check_name                              | severity | bad_rows
----------------------------------------+----------+---------
released dataset without citation fields | error    | 0
deaccessioned dataset                   | warn     | 2
draft version                           | warn     | 1
(3 rows)
```

SQL

```sql
-- UNION ALL BY NAME: match columns by name, fill the missing ones with NULL (schema drift).
SELECT 1 AS id, 'old file' AS source, 'x' AS title
UNION ALL BY NAME
SELECT 2 AS id, 'new file' AS source, 3 AS version;
```

Result

```
id | source   | title | version
---+----------+-------+--------
1  | old file | x     | NULL
2  | new file | NULL  | 3
(2 rows)
```

LESSON 07

CTEs, subqueries and recursion

Level: intermediate | In this project: `WITH scalars AS (...), pivoted AS (...)`, scalar subqueries in the last quality check | Needs: lessons 02, 04

1. Theory

A **CTE** (common table expression) names a query so later parts can use it like a table: `WITH a AS (...), b AS (SELECT ... FROM a) SELECT ... FROM b`. It makes long queries read top to bottom instead of inside out. Most engines (DuckDB included) inline a CTE, so it costs nothing extra compared with a subquery; the point is readability and reuse.

Subquery kinds:

- **Scalar subquery**: returns exactly one value; usable wherever a value is: `(SELECT max(x) FROM t)`.
- **Derived table**: `FROM (SELECT ...) AS name`.
- **IN / EXISTS subquery**: membership tests (`EXISTS` is safer with NULLs, lesson 03).
- **Correlated subquery**: refers to a column of the outer query, so it is evaluated "per outer row" (the optimizer usually turns it into a join).

A **recursive CTE** walks hierarchies and graphs: an *anchor* query gives the starting rows, a *recursive member* joins the CTE to itself to find the next level, and `UNION ALL` stacks the levels until no new rows appear. Always make sure the recursion can end (no cycles, or add a depth limit).

2. Real-world case

1. The flatten SQL builds `scalars` (columns straight from the JSON) and `pivoted` (fields turned into columns) as two CTEs, then joins them. Each piece can be tested on its own.
2. The last quality check compares two numbers from different tables with two scalar subqueries.
3. Dataverse collections form a tree (root > institution > unit > sub-unit). The extractor takes a flat comma-separated `ALL_DATAVERSE_LIST`. A recursive CTE turns "this collection and everything below it" into that list, so a new sub-collection is picked up without editing the list by hand.

3. Popular usage

- Step-by-step data cleaning pipelines in dbt models (`with source as (...), renamed as (...)`).
- Organisation charts, category trees, bill-of-materials, folder paths (recursive CTE).
- "Compare to the overall number" with a scalar subquery or a one-row CTE.

4. Example code

Python

```python
run_sql_file("setup_bronze.sql")   # bronze_meta: the raw sample records
```

SQL

```sql
-- A CTE chain: files per run, then each run's share of all files (the second CTE uses the first).
WITH per_run AS (
    SELECT run_id, sum(COALESCE(len(data.latestVersion.files), 0)) AS n_files
    FROM bronze_meta GROUP BY run_id
),
total AS (SELECT sum(n_files) AS all_files FROM per_run)
SELECT p.run_id, p.n_files, round(100.0 * p.n_files / t.all_files, 1) AS pct_of_all
FROM per_run p CROSS JOIN total t
ORDER BY p.run_id;
```

Result

```
run_id                  | n_files | pct_of_all
------------------------+---------+-----------
20261012T000000Z_aaaaaa | 2       | 33.3
20261013T000000Z_bbbbbb | 4       | 66.7
(2 rows)
```

SQL

```sql
-- Scalar subqueries: reconcile two counts from the same table.
SELECT (SELECT count(*) FROM bronze_meta)                                                    AS records,
       (SELECT count(DISTINCT data.identifier) FROM bronze_meta)                             AS distinct_datasets,
       (SELECT count(*) FROM bronze_meta) - (SELECT count(DISTINCT data.identifier) FROM bronze_meta) AS repeated;
```

Result

```
records | distinct_datasets | repeated
--------+-------------------+---------
6       | 4                 | 2
(1 row)
```

SQL

```sql
-- Correlated subquery: datasets with more files than the average of THEIR OWN run.
SELECT b.run_id, b.data.identifier AS id, len(b.data.latestVersion.files) AS n_files
FROM bronze_meta b
WHERE len(b.data.latestVersion.files) >
      (SELECT avg(len(x.data.latestVersion.files)) FROM bronze_meta x WHERE x.run_id = b.run_id)
ORDER BY 1, 2;
```

Result

```
run_id                  | id  | n_files
------------------------+-----+--------
20261012T000000Z_aaaaaa | DS1 | 2
20261013T000000Z_bbbbbb | DS1 | 3
(2 rows)
```

SQL

```sql
-- A hierarchy: each collection points at its parent.
CREATE TABLE dataverses AS
SELECT * FROM (VALUES
    ('root', NULL), ('brin', 'root'),
    ('orpertanian', 'brin'), ('perikanan', 'brin'), ('kehutanan', 'brin'),
    ('hutan_mangrove', 'kehutanan')
) AS t(alias, parent);
```

SQL

```sql
-- Recursive CTE: "brin and everything below it", with the depth and the path.
WITH RECURSIVE tree AS (
    SELECT alias, 0 AS depth, alias AS path FROM dataverses WHERE alias = 'brin'       -- anchor
    UNION ALL
    SELECT d.alias, t.depth + 1, t.path || ' > ' || d.alias                             -- next level
    FROM dataverses d JOIN tree t ON d.parent = t.alias
)
SELECT depth, path FROM tree ORDER BY path;
```

Result

```
depth | path
------+---------------------------------
0     | brin
1     | brin > kehutanan
2     | brin > kehutanan > hutan_mangrove
1     | brin > orpertanian
1     | brin > perikanan
(5 rows)
```

SQL

```sql
-- The same tree as a flat list, ready for the ALL_DATAVERSE_LIST setting.
WITH RECURSIVE tree AS (
    SELECT alias FROM dataverses WHERE alias = 'brin'
    UNION ALL
    SELECT d.alias FROM dataverses d JOIN tree t ON d.parent = t.alias
)
SELECT string_agg(alias, ',' ORDER BY alias) AS all_dataverse_list FROM tree;
```

Result

```
all_dataverse_list
-------------------------------------------------
brin,hutan_mangrove,kehutanan,orpertanian,perikanan
(1 row)
```

LESSON 08

Window functions

Level: intermediate | In this project: `ROW_NUMBER() OVER (PARTITION BY global_id ORDER BY ...)` for de-duplication | Needs: lesson 04

1. Theory

A window function computes a value for each row using a set of related rows (its "window"), WITHOUT collapsing them the way `GROUP BY` does. Every input row stays in the output.

```
function(...) OVER (PARTITION BY <group>  ORDER BY <order>  <frame>)
```

- `PARTITION BY` splits rows into independent groups (omit it: one group for the whole table).
- `ORDER BY` orders the rows inside each group (needed by ranking and offset functions).
- The **frame** says which rows around the current one are included, e.g. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. With `ORDER BY` and no frame, an aggregate window is a running total from the start to the current row.

Families:

- **Ranking**: `row_number()` (always unique 1,2,3), `rank()` (ties share a number, then a gap: 1,1,3), `dense_rank()` (ties share, no gap: 1,1,2), `ntile(n)` (buckets).
- **Offset**: `lag(x, n)` / `lead(x, n)` read the previous / next row; `first_value`, `last_value`.
- **Aggregate as window**: `sum(x) OVER (...)`, `avg`, `count`, `min`, `max`.
- `WINDOW w AS (...)` names a window so several functions can share it.

Windows run after `WHERE` / `GROUP BY` / `HAVING`, so filter on them with `QUALIFY` or a subquery (lesson 04).

2. Real-world case

Every extraction run contains a full copy of the catalogue, so the same dataset appears once per run. Questions that need "the other rows of the same dataset":

- "Did the number of files change since the previous run?" -> `lag()` over the dataset's runs.
- "Keep only the newest copy" -> `row_number()` and keep 1 (lesson 09; this is the silver dedup).
- "Running total of deposits per month, with a 3-month moving average" -> aggregate windows.
- "The two biggest datasets of each dataverse" -> `rank()` with `QUALIFY`.

3. Popular usage

- Dedup: "latest row per key" (the single most common use in data engineering).
- Month-over-month change (`lag`), running balances (`sum() OVER`), moving averages.
- Top-N per group, percentiles (`ntile`), sessionization (gaps between events with `lag`).
- Share of total: `x / sum(x) OVER ()`.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")   # bronze_meta: 6 dataset records over 2 runs
```

SQL

```
-- Compare each dataset's file count with the same dataset in the previous run (lag).
WITH v AS (
    SELECT data.identifier AS id, run_id, COALESCE(len(data.latestVersion.files), 0) AS n_files
    FROM bronze_meta
)
SELECT id, run_id, n_files,
       lag(n_files) OVER w              AS prev_files,
       n_files - lag(n_files) OVER w    AS change
FROM v
WINDOW w AS (PARTITION BY id ORDER BY run_id)
ORDER BY id, run_id;
```

Result

```
id  | run_id                  | n_files | prev_files | change
----+-------------------------+---------+------------+-------
DS1 | 20261012T000000Z_aaaaaa | 2       | NULL       | NULL
DS1 | 20261013T000000Z_bbbbbb | 3       | 2          | 1
DS2 | 20261012T000000Z_aaaaaa | 0       | NULL       | NULL
DS2 | 20261013T000000Z_bbbbbb | 0       | 0          | 0
DS3 | 20261013T000000Z_bbbbbb | 1       | NULL       | NULL
DS4 | 20261012T000000Z_aaaaaa | 0       | NULL       | NULL
(6 rows)
```

SQL

```
-- Ranking functions differ only on ties (DS1 and DS2 both have 10).
SELECT id, size,
       row_number() OVER (ORDER BY size DESC, id) AS row_number,
       rank()       OVER (ORDER BY size DESC)     AS rank,
       dense_rank() OVER (ORDER BY size DESC)     AS dense_rank
FROM (VALUES ('DS1', 10), ('DS2', 10), ('DS3', 7), ('DS4', 5)) AS t(id, size)
ORDER BY size DESC, id;
```

Result

```
id  | size | row_number | rank | dense_rank
----+------+------------+------+-----------
DS1 | 10   | 1          | 1    | 1
DS2 | 10   | 2          | 1    | 1
DS3 | 7    | 3          | 3    | 2
DS4 | 5    | 4          | 4    | 3
(4 rows)
```

SQL

```sql
-- Aggregate windows: running total, 3-month moving average, share of the total.
SELECT month, deposits,
       sum(deposits) OVER (ORDER BY month)                                          AS running_total,
       round(avg(deposits) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS moving_avg_3,
       round(100.0 * deposits / sum(deposits) OVER (), 1)                           AS pct_of_total
FROM (VALUES ('2025-01', 4), ('2025-02', 6), ('2025-03', 5), ('2025-04', 9)) AS t(month, deposits)
ORDER BY month;
```

Result

```
month   | deposits | running_total | moving_avg_3 | pct_of_total
--------+----------+---------------+--------------+-------------
2025-01 | 4        | 4             | 4.0          | 16.7
2025-02 | 6        | 10            | 5.0          | 25.0
2025-03 | 5        | 15            | 5.0          | 20.8
2025-04 | 9        | 24            | 6.7          | 37.5
(4 rows)
```

SQL

```sql
-- Top 2 datasets per dataverse: rank inside each partition, keep rank <= 2 with QUALIFY.
SELECT dataverse, id, n_files
FROM (VALUES ('orpertanian', 'DS1', 3), ('orpertanian', 'DS3', 1), ('orpertanian', 'DS2', 0),
             ('perikanan', 'DS6', 10), ('perikanan', 'DS7', 4)) AS t(dataverse, id, n_files)
QUALIFY rank() OVER (PARTITION BY dataverse ORDER BY n_files DESC) <= 2
ORDER BY dataverse, n_files DESC;
```

Result

```
dataverse   | id  | n_files
------------+-----+--------
orpertanian | DS1 | 3
orpertanian | DS3 | 1
perikanan   | DS6 | 10
perikanan   | DS7 | 4
(4 rows)
```

LESSON 09

Latest-version dedup with ROW_NUMBER and QUALIFY

Level: intermediate | In this project: `silver_datasets` (section 4 of `flatten_datasets_metadata.sql`) | Needs: lessons 04, 08

1. Theory

De-duplication pattern: number the copies of each key with `row_number()`, ordered so the copy you want is first, then keep number 1.

```
row_number() OVER (PARTITION BY <business key> ORDER BY <which copy wins> DESC, <tie-breakers>) = 1
```

Design decisions:

- **The key** (`PARTITION BY`): what makes two rows "the same thing" (here `global_id`).
- **Which copy wins**: prefer a business timestamp ("last updated at the source") over a load time, when you have one.
- **NULLs in the sort key**: say `NULLS LAST` explicitly so the result does not depend on an engine default.
- **Tie-breakers**: if two rows tie, `row_number()` picks one at random. Add more sort columns until the order is total, so every re-run gives the same answer.
- **Where to filter**: `QUALIFY` (DuckDB, Snowflake, BigQuery) or a subquery with `WHERE rn = 1` (everywhere). PostgreSQL also has `DISTINCT ON (key)`. DuckDB has `arg_max(value, order_col)` for "the value from the newest row".

2. Real-world case

Each extraction run exports the whole catalogue, so every dataset is repeated once per run. Counting rows overstates the catalogue; DS1 even has a different title in each run. `silver_datasets` must have ONE row per `global_id`, holding the newest version: newest `last_update_time`, then newest `run_id`, then `dataverse` as a last tie-breaker. The quality check `duplicate global_id in silver_datasets` proves it worked.

3. Popular usage

- "Latest record per customer / device / entity" from an append-only event or snapshot table (CDC feeds).
- Staging models in dbt that deduplicate a raw source.
- "First purchase per customer" (same pattern, ascending order).

4. Example code

Python

```python
run_sql_file("setup_bronze.sql")   # bronze_meta: 6 dataset records over 2 runs
```

SQL

```sql
-- Six rows for four datasets. updated is an instant (TRY_CAST to TIMESTAMPTZ handles all three formats).
CREATE TEMP TABLE versions AS
SELECT data.identifier AS id, run_id,
       TRY_CAST(data.latestVersion.lastUpdateTime AS TIMESTAMPTZ) AS updated
FROM bronze_meta;
SELECT * FROM versions ORDER BY id, run_id;
```

Result

```
id  | run_id                  | updated
----+-------------------------+----------------------
DS1 | 20261012T000000Z_aaaaaa | 2025-01-01 00:00:00+00
DS1 | 20261013T000000Z_bbbbbb | 2025-06-01 00:00:00+00
DS2 | 20261012T000000Z_aaaaaa | NULL
DS2 | 20261013T000000Z_bbbbbb | NULL
DS3 | 20261013T000000Z_bbbbbb | 2025-05-20 01:30:00+00
DS4 | 20261012T000000Z_aaaaaa | 2025-02-10 09:00:00+00
(6 rows)
```

SQL

```sql
-- QUALIFY: keep the newest copy per dataset. DS2 has no timestamp at all; the run_id tie-breaker still picks B.
SELECT * FROM versions
QUALIFY row_number() OVER (PARTITION BY id ORDER BY updated DESC NULLS LAST, run_id DESC) = 1
ORDER BY id;
```

Result

```
id  | run_id                  | updated
----+-------------------------+----------------------
DS1 | 20261013T000000Z_bbbbbb | 2025-06-01 00:00:00+00
DS2 | 20261013T000000Z_bbbbbb | NULL
DS3 | 20261013T000000Z_bbbbbb | 2025-05-20 01:30:00+00
DS4 | 20261012T000000Z_aaaaaa | 2025-02-10 09:00:00+00
(4 rows)
```

SQL

```sql
-- The portable form (works in PostgreSQL, SQL Server, ...): number in a subquery, filter outside.
SELECT id, run_id, updated
FROM (SELECT *, row_number() OVER (PARTITION BY id ORDER BY updated DESC NULLS LAST, run_id DESC) AS rn
      FROM versions)
WHERE rn = 1
ORDER BY id;
```

Result

```
id  | run_id                  | updated
----+-------------------------+----------------------
DS1 | 20261013T000000Z_bbbbbb | 2025-06-01 00:00:00+00
DS2 | 20261013T000000Z_bbbbbb | NULL
DS3 | 20261013T000000Z_bbbbbb | 2025-05-20 01:30:00+00
DS4 | 20261012T000000Z_aaaaaa | 2025-02-10 09:00:00+00
(4 rows)
```

SQL

```sql
-- arg_max(value, order_col) = "value from the row with the biggest order_col". Short, but it skips rows whose
-- order_col is NULL: DS2 becomes NULL here. Use it only when the order column is never NULL.
SELECT id, arg_max(run_id, updated) AS newest_run
FROM versions GROUP BY id ORDER BY id;
```

Result

```
id  | newest_run
----+-----------------------
DS1 | 20261013T000000Z_bbbbbb
DS2 | NULL
DS3 | 20261013T000000Z_bbbbbb
DS4 | 20261012T000000Z_aaaaaa
(4 rows)
```

SQL

```sql
-- Why tie-breakers matter: both rows have the same date. With run_id DESC added, the answer is always B.
SELECT id, run_id, updated
FROM (VALUES ('DS9', 'A', DATE '2025-01-01'), ('DS9', 'B', DATE '2025-01-01')) AS t(id, run_id, updated)
QUALIFY row_number() OVER (PARTITION BY id ORDER BY updated DESC, run_id DESC) = 1;
```

Result

```
id  | run_id | updated
----+--------+-----------
DS9 | B      | 2025-01-01
(1 row)
```

SQL

```sql
-- The proof, as the project's quality check does it: no key may appear twice after the dedup.
WITH kept AS (
    SELECT * FROM versions
    QUALIFY row_number() OVER (PARTITION BY id ORDER BY updated DESC NULLS LAST, run_id DESC) = 1
)
SELECT (SELECT count(*) FROM versions)                  AS rows_before,
       (SELECT count(*) FROM kept)                      AS rows_after,
       (SELECT count(*) FROM (SELECT id FROM kept GROUP BY id HAVING count(*) > 1)) AS duplicate_keys;
```

Result

```
rows_before | rows_after | duplicate_keys
------------+------------+---------------
6           | 4          | 0
(1 row)
```

PostgreSQL (shown, not run)

```sql
-- PostgreSQL only: DISTINCT ON keeps the first row of each key in the ORDER BY order. (Not run here.)
SELECT DISTINCT ON (id) id, run_id, updated
FROM versions
ORDER BY id, updated DESC NULLS LAST, run_id DESC;
```

LESSON 10

Reading JSON files: schema-on-read and hive partitioning

Level: intermediate | In this project: section 1 of `flatten_datasets_metadata.sql` (`read_json(... columns = {...})`) | Needs: nothing

1. Theory

Schema-on-write (a normal database table): the structure is enforced when data is stored. **Schema-on-read** (files in a lake): anything can be stored, and the structure is applied when you read. The benefit is that ingestion never fails because the source changed. The cost is that reading, typing and validating are now your job.

DuckDB's `read_json` can work two ways:

- `read_json_auto(path)` samples the file and GUESSES the columns and types. Fast to start, but the guess can change when the data changes (a column that was always a number meets one text value).
- `read_json(path, columns = {name: 'TYPE', ...})` DECLARES the shape you depend on. Keys missing from a record become NULL, keys you did not list are ignored, and a value of the wrong type fails loudly. This is "schema on read" done deliberately.

Other options you will use: `format = 'newline_delimited'` (one JSON document per line, JSONL), `.gz` files are decompressed automatically, a LIST of paths or a glob reads many files at once, and `filename = true` adds a column with the source file.

Hive partitioning: folder names of the form `key=value` (`run_id=20261012T...` , `dataverse=orpertanian`) become columns when you pass `hive_partitioning = true` . DuckDB guesses their types (a date-looking folder becomes DATE; a number-looking one becomes a number), so check them.

2. Real-world case

The Dataverse API output is not uniform: a DRAFT version has no `versionNumber` or `releaseTime` , a deaccessioned dataset has no `latestVersion` at all, and a server upgrade can add keys. The flatten script therefore declares exactly the STRUCT it needs. Everything else in the raw files is kept in bronze but ignored here. The run id, extract date and dataverse come from the folder names, so no column has to be added to the JSON when it is stored.

3. Popular usage

- Querying a data lake of JSON / CSV / Parquet files directly with SQL, no load step.
- Landing zones where the source may change: store raw, declare the schema at read time.
- Partitioned layouts (`year=2026/month=10/...`) so engines can skip folders (lesson 23).

4. Example code

SQL

```sql
-- Auto-detect: DuckDB guesses the top-level columns. (Hive columns come from the folder names.)
SELECT column_name, column_type
FROM (DESCRIBE SELECT * FROM read_json_auto(
    'sample_data/extract_date=*/run_id=*/entity=dataset_metadata/dataverse=*/*.jsonl',
    hive_partitioning = true));
```

Result

```
column_name  | column_type
-------------+-----------------------------------------------------------------
status       | VARCHAR
data         | STRUCT(id BIGINT, identifier VARCHAR, protocol VARCHAR, autho...
dataverse    | VARCHAR
entity       | VARCHAR
extract_date | DATE
run_id       | VARCHAR
(6 rows)
```

SQL

```sql
-- Declared schema (a short version of the project's): only the keys we depend on.
-- DRAFT and deaccessioned datasets simply produce NULLs.
SELECT run_id, extract_date, dataverse,
       data.identifier                      AS id,
       data.latestVersion.versionState      AS state,
       data.latestVersion.versionNumber     AS version
FROM read_json(
    'sample_data/extract_date=*/run_id=*/entity=dataset_metadata/dataverse=*/*.jsonl',
    format = 'newline_delimited',
    hive_partitioning = true,
    columns = {
        status: 'VARCHAR',
        data: 'STRUCT(id BIGINT, identifier VARCHAR, publicationDate VARCHAR,
                      latestVersion STRUCT(versionState VARCHAR, versionNumber INTEGER, lastUpdateTime VARCHAR))'
    })
ORDER BY run_id, id;
```

Result

```
run_id                  | extract_date | dataverse   | id  | state    | version
------------------------+--------------+-------------+-----+----------+--------
20261012T000000Z_aaaaaa | 2026-10-12   | orpertanian | DS1 | RELEASED | 1
20261012T000000Z_aaaaaa | 2026-10-12   | orpertanian | DS2 | NULL     | NULL
20261012T000000Z_aaaaaa | 2026-10-12   | orpertanian | DS4 | DRAFT    | NULL
20261013T000000Z_bbbbbb | 2026-10-13   | orpertanian | DS1 | RELEASED | 1
20261013T000000Z_bbbbbb | 2026-10-13   | orpertanian | DS2 | NULL     | NULL
20261013T000000Z_bbbbbb | 2026-10-13   | orpertanian | DS3 | RELEASED | 1
(6 rows)
```

SQL

```sql
-- filename = true records which file each row came from (lineage). A list of files works like a glob;
-- silver.py passes exactly the files named by the run manifests.
SELECT DISTINCT regexp_extract(filename, 'run_id=[^/]+', 0) AS run_folder, count(*) AS records
FROM read_json(
    ['sample_data/extract_date=2026-10-12/run_id=20261012T000000Z_aaaaaa/entity=dataset_metadata/dataverse=orpertanian/part-00000.jsonl'],
    format = 'newline_delimited', filename = true,
    columns = {status: 'VARCHAR'})
GROUP BY ALL;
```

Result

```
run_folder                     | records
-------------------------------+--------
run_id=20261012T000000Z_aaaaaa | 3
(1 row)
```

Python

```python
# Drift: line 3 has text where a number is declared. A declared schema fails LOUDLY (good: you notice).
import pathlib, tempfile
tmp = pathlib.Path(tempfile.mkdtemp())
path = tmp / "drift.jsonl"
path.write_text('{"id": 1, "n": 5, "extra": "x"}\n{"id": 2}\n{"id": 3, "n": "oops"}\n', encoding="utf-8")

try:
    con.sql(f"SELECT * FROM read_json('{path.as_posix()}', format='newline_delimited', "
            f"columns={{id: 'BIGINT', n: 'INTEGER'}})").fetchall()
except duckdb.Error as e:
    print("typed read failed ->", str(e).splitlines()[0].split(", in ")[-1][:100])

# The tolerant alternative: land the risky column as text, then TRY_CAST and count what failed (lesson 18).
rel = con.sql(f"SELECT * FROM read_json('{path.as_posix()}', format='newline_delimited', "
              f"columns={{id: 'BIGINT', n: 'VARCHAR'}})")
print(rel.project("id, n, TRY_CAST(n AS INTEGER) AS n_int").fetchall())
```

Result

```
typed read failed -> line 3: Failed to cast value to numerical: "oops"
[(1, '5', 5), (2, None, None), (3, 'oops', None)]
```

LESSON 11

Nested types: STRUCT, LIST, MAP and JSON

Level: intermediate | In this project: the `data STRUCT(...)` column in `bronze_meta`; `files STRUCT(...)[]` | Needs: lesson 10

1. Theory

Modern analytic engines store nested data natively, so you do not have to flatten everything at load time.

Type	What it is	Access	Literal
`STRUCT`	a record: fixed named fields, each with its own type	`s.field` (dot)	`{'a': 1, 'b': 'x'}`
`LIST` (array)	any number of values of ONE type	`l[1]` (1-based!), `l[-1]` (last), `l[1:2]` (slice), `len(l)`	`[1, 2, 3]`
`MAP`	key/value pairs with any keys	`m['key']`	`MAP {'k': 'v'}`
`JSON`	text that is valid JSON; shape is free	`->`, `->>` (lesson 12)	`'{"a":1}'::JSON`

Rules of thumb:

- A STRUCT or LIST column can contain other STRUCTs and LISTs: `STRUCT(...)[]` is a list of records.
- Reaching into a NULL struct gives NULL (no error). Reaching past the end of a list gives NULL too.
- Use STRUCT / LIST when the shape is known and stable (fast, typed). Use JSON when the shape varies per row or you do not control it.
- Lists are 1-based in DuckDB (and in standard SQL arrays), unlike most programming languages.

2. Real-world case

One dataset record is a document: `data.latestVersion.license.name`, `data.latestVersion.files[...]`, `data.latestVersion.metadataBlocks.citation.fields[...]`. Questions like "license name", "number of files" or "name of the first file" do not need any join if you can walk into the structure with dots and `len()`. The project keeps the nesting in `bronze_meta` and only flattens when a question needs rows (lesson 13). The field `value` is JSON because in the API it is sometimes text, sometimes a list, sometimes an object.

3. Popular usage

- Event and log data (nested payloads), API responses, Parquet files written by Spark / BigQuery.
- Returning several related values from one expression (`struct_pack`) without a helper table.
- Pre-aggregating a child table into a LIST column to avoid a join in dashboards.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")   # bronze_meta.data is a STRUCT with nested STRUCTs and LISTs
```

SQL

```
-- Walk into the structure with dots; len() counts list items. DS2 (deaccessioned) has no latestVersion: all NULL.
SELECT data.identifier                                AS id,
       data.latestVersion.license.name                AS license,
       len(data.latestVersion.files)                  AS n_files,
       data.latestVersion.files[1].dataFile.filename  AS first_file,
       data.latestVersion.files[-1].dataFile.filename AS last_file
FROM bronze_meta
WHERE run_id LIKE '%bbbbbb'
ORDER BY id;
```

Result

```
id  | license | n_files | first_file  | last_file
----+---------+---------+-------------+-------------
DS1 | CC0 1.0 | 3       | data.csv    | codebook.pdf
DS2 | NULL    | NULL    | NULL        | NULL
DS3 | CC0 1.0 | 1       | landing.csv | landing.csv
(3 rows)
```

SQL

```
-- typeof() shows the type of an expression: handy to learn an unfamiliar structure.
SELECT typeof(data.latestVersion.license)                      AS license_type,
       typeof(data.latestVersion.files) LIKE '%[]'             AS files_is_a_list,
       typeof(data.latestVersion.files[1].dataFile)            AS one_file_type
FROM bronze_meta LIMIT 1;
```

Result

```
license_type                       | files_is_a_list | one_file_type
-----------------------------------+-----------------+-----------------------------------------------------------------
STRUCT("name" VARCHAR, uri VARCHAR) | true            | STRUCT(id BIGINT, filename VARCHAR, contentType VARCHAR, file...
(1 row)
```

SQL

```
-- A STRUCT column can be expanded into one column per field with ".*" (name the struct first).
SELECT id, license.*
FROM (SELECT data.identifier AS id, data.latestVersion.license AS license
      FROM bronze_meta WHERE run_id LIKE '%bbbbbb')
ORDER BY id;
```

Result

```
id  | name    | uri
----+---------+-------------------------------------------------
DS1 | CC0 1.0 | http://creativecommons.org/publicdomain/zero/1.0
DS2 | NULL    | NULL
DS3 | CC0 1.0 | http://creativecommons.org/publicdomain/zero/1.0
(3 rows)
```

SQL

```
-- Literals: build STRUCT, LIST and MAP values by hand, and index into them.
SELECT s,
       s.a            AS struct_field,
       l[2]           AS second_item,
       l[2:3]         AS slice,
       m['k']         AS map_value
FROM (SELECT {'a': 1, 'b': 'x'} AS s, [10, 20, 30] AS l, MAP {'k': 'v'} AS m);
```

Result

```
s               | struct_field | second_item | slice    | map_value
----------------+--------------+-------------+----------+----------
{'a': 1, 'b': x} | 1            | 20          | [20, 30] | v
(1 row)
```

SQL

```sql
-- Build a STRUCT from columns, and a LIST from rows (list() is an aggregate).
SELECT run_id,
       list(data.identifier ORDER BY data.identifier)                                  AS ids,
       struct_pack(n_records := count(*), first_id := min(data.identifier))            AS summary
FROM bronze_meta GROUP BY run_id ORDER BY run_id;
```

Result

```
run_id                 | ids             | summary
-----------------------+-----------------+---------------------------------
20261012T000000Z_aaaaaa | [DS1, DS2, DS4] | {'n_records': 3, 'first_id': DS1}
20261013T000000Z_bbbbbb | [DS1, DS2, DS3] | {'n_records': 3, 'first_id': DS1}
(2 rows)
```

SQL

```sql
-- JSON is for free-shaped values. The title field arrives as JSON text; casting shows the difference.
SELECT '"Rice yield"'::JSON AS json_value, ('"Rice yield"'::JSON)::VARCHAR AS as_text_with_quotes,
       json_type('"Rice yield"'::JSON) AS json_type;
```

Result

```
json_value   | as_text_with_quotes | json_type
-------------+---------------------+----------
"Rice yield" | "Rice yield"        | VARCHAR
(1 row)
```

LESSON 12

JSON functions: -> ->> JSONPath, json_type, from_json

Level: intermediate | In this project: `value ->> '$'`, `item ->> '$.authorName.value'`, `from_json(f.value, '["JSON"]')` | Needs: lesson 11

1. Theory

A JSON value has six possible shapes: object `{}`, array `[]`, string, number, boolean and `null`. You navigate it with a path written in JSONPath notation:

Path	Means
`$`	the whole value
`$.key`	field `key` of an object
`$[0]`	first element of an array (JSON arrays are 0-based, unlike SQL lists)
`$.a.b[1].c`	chain them
`$[*]`	every element (returns a list)

Operators and functions:

- `j -> path` returns JSON (keeps quotes around strings). `j ->> path` returns plain text (VARCHAR). `->> '$'` on a string unwraps it: `'"abc"'::JSON ->> '$'` is `abc`.
- `json_extract(j, path)` and `json_extract_string(j, path)` are the function forms.
- `json_type(j)` says what it is (`OBJECT`, `ARRAY`, `VARCHAR`, `BIGINT`, `NULL`, ...). Use it to branch on shape.
- `json_array_length(j)`, `json_keys(j)` inspect arrays and objects.
- `from_json(j, 'TYPE')` converts JSON into a typed value: `from_json(j, '["VARCHAR"]')` gives a `VARCHAR[]`; `'["JSON"]'` gives a list of JSON values you can `UNNEST` (lesson 13).
- A missing key does not fail: it returns SQL NULL. A JSON `null` is a real value, NOT a SQL NULL (see below).

2. Real-world case

The Dataverse field `value` has three shapes depending on the field:

- `title`, `depositor`: a plain string.
- `subject`: a list of strings.
- `author`, `keyword`, `grantNumber`, `dsDescription`: a list of objects whose real text sits one level deeper, in `.authorName.value`.

The project reads each shape with the matching call: `->> '$'` for strings, `from_json(..., '["VARCHAR"]')` for string lists, `->> '$.authorName.value'` inside each object of a list. Checking `json_type` first tells you which one applies.

3. Popular usage

- Semi-structured event payloads, API logs, webhook bodies stored in a JSON column (Postgres `jsonb`, BigQuery, Snowflake `VARIANT`).
- Pulling a few attributes out of a JSON blob into real columns during staging.
- Schema discovery: `json_type` and `json_keys` on unknown data.

4. Example code

Python

```python
run_sql_file("setup_bronze.sql")
run_sql_file("setup_fields.sql")   # stg_fields: one row per dataset field, with a JSON "value" column
```

SQL

```sql
-- What shape does each field have? (Run it before writing extraction code.)
SELECT type_name, json_type(value) AS shape, count(*) AS n
FROM stg_fields GROUP BY ALL ORDER BY type_name, shape;
```

Result

```
type_name     | shape   | n
--------------+---------+--
author        | ARRAY   | 4
dateOfDeposit | VARCHAR | 4
depositor     | VARCHAR | 4
dsDescription | ARRAY   | 3
grantNumber   | ARRAY   | 2
keyword       | ARRAY   | 3
subject       | ARRAY   | 4
title         | VARCHAR | 4
(8 rows)
```

SQL

```sql
-- -> keeps JSON (strings keep their quotes). ->> returns text. ->> '$' unwraps a JSON string.
SELECT value                 AS as_json,
       value::VARCHAR        AS cast_text,
       value ->> '$'         AS unwrapped
FROM stg_fields
WHERE type_name = 'title' AND run_id LIKE '%bbbbbb'
ORDER BY global_id;
```

Result

```
as_json                 | cast_text               | unwrapped
------------------------+-------------------------+----------------------
"Rice yield survey 2025" | "Rice yield survey 2025" | Rice yield survey 2025
"Fish landing records"   | "Fish landing records"   | Fish landing records
(2 rows)
```

SQL

```sql
-- Paths into an array of objects: JSONPath arrays are 0-based. [*] returns all elements as a list.
SELECT global_id,
       json_array_length(value)                          AS n_authors,
       value ->> '$[0].authorName.value'                 AS first_author,
       value ->> '$[1].authorName.value'                 AS second_author,
       json_extract_string(value, '$[*].authorName.value') AS all_authors
FROM stg_fields
WHERE type_name = 'author' AND run_id LIKE '%bbbbbb'
ORDER BY global_id;
```

Result

```
global_id        | n_authors | first_author | second_author | all_authors
-----------------+-----------+--------------+---------------+--------------------------
doi:10.99999/DS1 | 2         | Doe, Jane    | Roe, Rick     | ['Doe, Jane', 'Roe, Rick']
doi:10.99999/DS3 | 1         | Kim, Ana     | NULL          | ['Kim, Ana']
(2 rows)
```

SQL

```sql
-- from_json turns a JSON array of strings into a real typed list.
SELECT global_id, from_json(value, '["VARCHAR"]') AS subjects, len(from_json(value, '["VARCHAR"]')) AS n_subjects
FROM stg_fields
WHERE type_name = 'subject' AND run_id LIKE '%bbbbbb'
ORDER BY global_id;
```

Result

```
global_id        | subjects                                                        | n_subjects
-----------------+-----------------------------------------------------------------+-----------
doi:10.99999/DS1 | [Agricultural Sciences]                                         | 1
doi:10.99999/DS3 | [Agricultural Sciences, Earth and Environmental Sciences] | 2
(2 rows)
```

SQL

```sql
-- A missing key gives SQL NULL; a JSON null is a value of its own.
SELECT ('{"a": 1}'::JSON ->> '$.b') IS NULL      AS missing_key_is_sql_null,
       ('null'::JSON) IS NULL                    AS json_null_is_sql_null,
       '[' || json_type('null'::JSON) || ']'     AS json_null_type_as_text,
       ('{"a": null}'::JSON ->> '$.a') IS NULL   AS extracted_json_null_is_sql_null;
```

Result

```
missing_key_is_sql_null | json_null_is_sql_null | json_null_type_as_text | extracted_json_null_is_sql_null
------------------------+-----------------------+------------------------+--------------------------------
true                    | false                 | [NULL]                 | true
(1 row)
```

LESSON 13

UNNEST: turning a list into rows

Level: intermediate | In this project: `UNNEST(...fields) AS t(f)` and `CROSS JOIN UNNEST(from_json(...)) WITH ORDINALITY` | Needs: lessons 11, 12

1. Theory

`UNNEST(list)` produces one output row per list element. It is how nested data becomes relational: a dataset with 7 fields becomes 7 rows.

Forms:

- In `SELECT`: `SELECT unnest([10, 20, 30])` gives three rows.
- In `FROM`, correlated with an earlier table: `FROM t, UNNEST(t.list_col) AS u(item)`. The `UNNEST` is evaluated once per row of `t` (a *lateral* join, with no `LATERAL` keyword needed in DuckDB).
- `WITH ORDINALITY` adds the 1-based position of each element as an extra column: `UNNEST(l) WITH ORDINALITY AS u(item, pos)`.
- Unnesting a LIST of STRUCTs gives a struct per row; read its fields with dots: `item.field`.

Two properties you must know:

1. **The grain changes.** After the `UNNEST` one row means "one element", not "one parent". Parent columns repeat on every element row, so summing a parent column now over-counts (lesson 02, fan-out).
2. **A NULL or empty list produces ZERO rows, and the parent row disappears.** To keep parents without elements use `LEFT JOIN UNNEST(...) AS u(item) ON true`; the element columns are then NULL.

The reverse operation is the aggregate `list(x)` (or `array_agg`), which collects rows back into a list.

2. Real-world case

`metadataBlocks.citation.fields` is a LIST of {typeName, value} entries, and some values are themselves lists (authors, subjects, keywords). Two `UNNEST` steps produce `stg_fields` (one row per field) and then the child tables (one row per author, with `WITH ORDINALITY` supplying `author_position` so the order from the source is kept).

The same property explains an observation in the project: deaccessioned datasets have no `latestVersion`, so their field list is NULL, so they produce no rows in `stg_fields` and no children. That is intended, but you have to know it is happening. `LEFT JOIN UNNEST` is the tool when you must not lose the parent.

3. Popular usage

- Exploding tags, line items, events or API arrays into rows for filtering and joining.
- Splitting delimited text: `UNNEST(string_split(tags, ';'))`.
- Recreating a "numbers table" or date series: `UNNEST(generate_series(...))`.
- PostgreSQL has the same idea (`unnest(array)`, also `WITH ORDINALITY`); Spark calls it `explode`, BigQuery `UNNEST`.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")   # bronze_meta: raw records, nested lists inside
```

SQL

```
-- Smallest example: one list becomes three rows.
SELECT unnest([10, 20, 30]) AS x;
```

Result

```
x
--
10
20
30
(3 rows)
```

SQL

```
-- Explode the field list: one row per (dataset, field). Note that DS2 (deaccessioned) is missing from the result.
SELECT data.identifier AS id, count(*) AS n_fields
FROM bronze_meta, UNNEST(data.latestVersion.metadataBlocks.citation.fields) AS t(f)
WHERE run_id LIKE '%bbbbbb'
GROUP BY id ORDER BY id;
```

Result

```
id  | n_fields
----+---------
DS1 | 7
DS3 | 8
(2 rows)
```

SQL

```
-- LEFT JOIN UNNEST ... ON true keeps the parent: DS2 now appears with 0 fields.
SELECT data.identifier AS id, count(f.typeName) AS n_fields
FROM bronze_meta
LEFT JOIN UNNEST(data.latestVersion.metadataBlocks.citation.fields) AS t(f) ON true
WHERE run_id LIKE '%bbbbbb'
GROUP BY id ORDER BY id;
```

Result

```
id  | n_fields
----+---------
DS1 | 7
DS2 | 0
DS3 | 8
(3 rows)
```

SQL

```sql
-- This is how the project builds stg_fields: one row per field, with the JSON value kept as JSON.
CREATE TEMP TABLE stg_fields AS
SELECT run_id,
       data.latestVersion.datasetPersistentId AS global_id,
       f.typeName AS type_name,
       f."value"  AS value
FROM bronze_meta, UNNEST(data.latestVersion.metadataBlocks.citation.fields) AS t(f);
SELECT type_name, count(*) AS n FROM stg_fields GROUP BY type_name ORDER BY type_name;
```

Result

```
type_name     | n
--------------+--
author        | 4
dateOfDeposit | 4
depositor     | 4
dsDescription | 3
grantNumber   | 2
keyword       | 3
subject       | 4
title         | 4
(8 rows)
```

SQL

```sql
-- Second level: the "author" value is a JSON array. UNNEST it WITH ORDINALITY to keep the source order.
SELECT f.global_id,
       a.pos                                  AS author_position,
       a.item ->> '$.authorName.value'        AS author_name,
       a.item ->> '$.authorAffiliation.value' AS affiliation
FROM stg_fields f
CROSS JOIN UNNEST(from_json(f.value, '["JSON"]')) WITH ORDINALITY AS a(item, pos)
WHERE f.type_name = 'author' AND f.run_id LIKE '%bbbbbb'
ORDER BY f.global_id, a.pos;
```

Result

```
global_id        | author_position | author_name | affiliation
-----------------+-----------------+-------------+------------
doi:10.99999/DS1 | 1               | Doe, Jane   | BRIN
doi:10.99999/DS1 | 2               | Roe, Rick   | NULL
doi:10.99999/DS3 | 1               | Kim, Ana    | BRIN
(3 rows)
```

SQL

```sql
-- A list of STRUCTs: one row per file, struct fields read with dots.
SELECT data.identifier AS id, f.dataFile.filename AS filename, f.restricted, f.dataFile.filesize AS bytes
FROM bronze_meta, UNNEST(data.latestVersion.files) AS t(f)
WHERE run_id LIKE '%bbbbbb'
ORDER BY id, filename;
```

Result

```
id  | filename     | restricted | bytes
----+--------------+------------+------
DS1 | codebook.pdf | false      | 4096
DS1 | data.csv     | false      | 10240
DS1 | notes.pdf    | true       | 2048
DS3 | landing.csv  | false      | 51200
(4 rows)
```

SQL

```sql
-- Delimited text to rows, and back to a list with list().
WITH tags AS (
    SELECT id, trim(tag) AS tag
    FROM (VALUES ('DS1', 'rice; yield'), ('DS3', 'fish')) AS t(id, tags),
         UNNEST(string_split(tags, ';')) AS u(tag)
)
SELECT id, count(*) AS n_tags, list(tag ORDER BY tag) AS tag_list FROM tags GROUP BY id ORDER BY id;
```

Result

```
id  | n_tags | tag_list
----+--------+-------------
DS1 | 2      | [rice, yield]
DS3 | 1      | [fish]
(2 rows)
```

SQL

```sql
-- Grain check after an UNNEST: rows produced must equal the total of the list lengths.
SELECT (SELECT count(*) FROM bronze_meta, UNNEST(data.latestVersion.files) AS t(f)) AS rows_after_unnest,
       (SELECT sum(len(data.latestVersion.files)) FROM bronze_meta)                 AS sum_of_list_lengths;
```

Result

```
rows_after_unnest | sum_of_list_lengths
------------------+--------------------
6                 | 6
(1 row)
```

LESSON 14

EAV, pivot and unpivot

Level: intermediate | In this project: the `pivoted` CTE (`max(...) FILTER (WHERE type_name = 'title')`) | Needs: lessons 05, 12

1. Theory

Two shapes for the same information:

```
WIDE (one column per attribute)            LONG / EAV (one row per attribute)
global_id | title | depositor                global_id | attribute | value
DS1       | ...   | ...                     DS1       | title     | ...
                                            DS1       | depositor | ...
```

EAV (entity-attribute-value) is flexible: new attributes need no schema change. It is hard to query: filtering on two attributes needs two lookups. So analysis tables are usually WIDE.

Pivot (long to wide): group by the entity and take, for each attribute, the value of the matching row. Three ways to write it:

1. **Conditional aggregation** (portable): `max(value) FILTER (WHERE attribute = 'title') AS title`, or `max(CASE WHEN attribute = 'title' THEN value END)`. `max()` is just a way to pick the single matching value.
2. `PIVOT` **statement** (DuckDB, Snowflake, Oracle, SQL Server): `PIVOT t ON attribute USING max(value) GROUP BY entity`. Convenient, but the output columns come from the data.
3. **Join the table to itself** once per attribute: works everywhere but is long and slow. Avoid.

Unpivot (wide to long) does the reverse: `UNPIVOT wide ON col1, col2 INTO NAME attribute VALUE value`.

Pitfall: a pivot silently hides duplicates. If an entity has TWO rows for `title`, `max()` quietly picks one. Check the grain first: (entity, attribute) must be unique, otherwise the pivot is wrong.

2. Real-world case

The Dataverse citation block is EAV: a list of {typeName, value}. The silver table needs `title`, `depositor` and `description` as columns so analysts can filter and group on them. The flatten script pivots only the SINGLE-valued fields into columns, and sends the MULTI-valued ones (authors, subjects, ...) to child tables (lesson 01). Rule of thumb: single-valued attribute -> column, multi-valued -> child table.

3. Popular usage

- Survey and form answers stored as rows, pivoted into one row per respondent.
- Key/value "settings" or "custom fields" tables turned into wide reporting tables.
- Month columns for finance-style reports (`PIVOT ... ON month`), and UNPIVOT to feed charts that want long data.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")
run_sql_file("setup_fields.sql")   # stg_fields: the long (EAV) table, one row per dataset field
```

SQL

```
-- The long shape for one dataset in run B: one row per field.
SELECT type_name, type_class, left(value::VARCHAR, 45) AS value_preview
FROM stg_fields
WHERE global_id = 'doi:10.99999/DS1' AND run_id LIKE '%bbbbbb'
ORDER BY type_name;
```

Result

```
type_name     | type_class           | value_preview
--------------+----------------------+---------------------------------------------
author        | compound             | [{"authorName":{"typeName":"authorName","mult
dateOfDeposit | primitive            | "2024-12-19"
depositor     | primitive            | "Doe, Jane"
dsDescription | compound             | [{"dsDescriptionValue":{"typeName":"dsDescrip
keyword       | compound             | [{"keywordValue":{"typeName":"keywordValue","
subject       | controlledVocabulary | ["Agricultural Sciences"]
title         | primitive            | "Rice yield survey 2025"
(7 rows)
```

SQL

```
-- Grain check BEFORE pivoting: (run, dataset, field) must be unique. Zero rows = safe to pivot.
SELECT run_id, global_id, type_name, count(*) AS n
FROM stg_fields GROUP BY ALL HAVING count(*) > 1;
```

Result

```
run_id | global_id | type_name | n
-------+-----------+-----------+--
(0 rows)
```

SQL

```
-- Pivot with conditional aggregation (the project's way): one row per (run, dataset).
SELECT run_id, global_id,
       max(value ->> '$') FILTER (WHERE type_name = 'title')         AS title,
       max(value ->> '$') FILTER (WHERE type_name = 'depositor')     AS depositor,
       max(value ->> '$') FILTER (WHERE type_name = 'dateOfDeposit') AS date_of_deposit
FROM stg_fields
GROUP BY run_id, global_id
ORDER BY global_id, run_id;
```

Result

```
run_id                  | global_id        | title                  | depositor | date_of_deposit
------------------------+------------------+------------------------+-----------+----------------
20261012T000000Z_aaaaaa | doi:10.99999/DS1 | Rice yield survey      | Doe, Jane | 2024-12-19
20261013T000000Z_bbbbbb | doi:10.99999/DS1 | Rice yield survey 2025 | Doe, Jane | 2024-12-19
20261013T000000Z_bbbbbb | doi:10.99999/DS3 | Fish landing records   | Kim, Ana  | 2025-05-19
20261012T000000Z_aaaaaa | doi:10.99999/DS4 | Soil moisture sensors  | Lee, Min  | 2025-02-01
(4 rows)
```

SQL

```sql
-- The same result with DuckDB's PIVOT statement. The attribute values become the column names.
PIVOT (SELECT run_id, global_id, type_name, value ->> '$' AS v
       FROM stg_fields WHERE type_name IN ('title', 'depositor'))
ON type_name USING max(v)
GROUP BY run_id, global_id
ORDER BY global_id, run_id;
```

Result

```
run_id                  | global_id        | depositor | title
------------------------+------------------+-----------+----------------------
20261012T000000Z_aaaaaa | doi:10.99999/DS1 | Doe, Jane | Rice yield survey
20261013T000000Z_bbbbbb | doi:10.99999/DS1 | Doe, Jane | Rice yield survey 2025
20261013T000000Z_bbbbbb | doi:10.99999/DS3 | Kim, Ana  | Fish landing records
20261012T000000Z_aaaaaa | doi:10.99999/DS4 | Lee, Min  | Soil moisture sensors
(4 rows)
```

SQL

```sql
-- UNPIVOT: wide back to long (for example to feed a chart or to store as EAV).
UNPIVOT (SELECT 'DS1' AS id, 'Rice yield survey' AS title, 'Doe, Jane' AS depositor)
ON title, depositor
INTO NAME attribute VALUE value;
```

Result

```
id  | attribute | value
----+-----------+------------------
DS1 | title     | Rice yield survey
DS1 | depositor | Doe, Jane
(2 rows)
```

SQL

```sql
-- Why the grain check matters: an entity with TWO titles. max() hides one of them without any warning.
SELECT id,
       max(v) FILTER (WHERE attribute = 'title')           AS title_shown,
       count(*) FILTER (WHERE attribute = 'title')         AS titles_found
FROM (VALUES ('DS1', 'title', 'Alpha'), ('DS1', 'title', 'Beta'), ('DS1', 'depositor', 'Doe')) AS t(id, attribute, v)
GROUP BY id;
```

Result

```
id  | title_shown | titles_found
----+-------------+-------------
DS1 | Beta        | 2
(1 row)
```

LESSON 15

Lambdas and list functions

Level: intermediate | In this project: `len(list_filter(data.latestVersion.files, f -> f.restricted))` | Needs: lesson 11

1. Theory

A **lambda** is a tiny unnamed function you pass to another function: `x -> x * 2` means "given x, return x times 2". DuckDB list functions accept lambdas to work on every element of a LIST without turning it into rows:

Function	Does
`list_transform(l, x -> expr)`	map: a new list with `expr` applied to each element
`list_filter(l, x -> cond)`	keep the elements where `cond` is true
`list_reduce(l, (a, b) -> expr)`	fold the list into one value
`list_sum`, `list_min`, `list_max`, `list_avg`, `list_aggregate(l, 'name')`	aggregate a list directly
`list_sort`, `list_distinct`, `list_contains(l, v)`, `list_concat`, `len(l)`	the everyday helpers
`array_to_string(l, sep)`, `string_split(s, sep)`	list to text and back

Syntax: the arrow form `x -> expr` is the long-standing one (this project uses it). Recent DuckDB versions also accept the Python-style `lambda x: expr`, which avoids confusion with the JSON `->` operator. Both work in the DuckDB installed here.

When to use which approach:

- **Lambda / list function**: the answer is one value per parent row (a count, a sum, a flag). It keeps the grain. No fan-out, no GROUP BY.
- **UNNEST + GROUP BY** (lesson 13): you need to filter, join or group the elements, or add them to other tables.

2. Real-world case

For each dataset the silver table needs `n_files` and `n_restricted_files`. The files are a LIST of structs inside the record. Instead of exploding all files into rows and grouping back (which loses datasets with no files unless you use a LEFT JOIN, lesson 13), the SQL counts with `len(list_filter(files, f -> f.restricted))`: one expression, one row per dataset, empty lists give 0.

3. Popular usage

- Counting or summing inside nested arrays (items in an order, tags on a post).
- Cleaning lists: `list_distinct(list_sort(...))`, removing NULLs with `list_filter(l, x -> x IS NOT NULL)`.
- Building display strings: `array_to_string(list_transform(...), ', ')`.
- The same idea exists as `transform`, `filter`, `aggregate` in Spark / Trino / Presto SQL.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")
```

SQL

```
-- Literals first: map, filter and fold on plain lists.
SELECT list_transform([1, 2, 3], x -> x * 10)         AS times_ten,
       list_filter([1, 2, 3, 4], x -> x % 2 = 0)      AS evens,
       list_reduce([1, 2, 3, 4], (a, b) -> a * b)     AS product;
```

Result

```
times_ten    | evens  | product
-------------+--------+--------
[10, 20, 30] | [2, 4] | 24
(1 row)
```

SQL

```
-- The project's pattern: count restricted files without exploding. DS2 (no latestVersion) gives NULL.
SELECT data.identifier AS id,
       len(data.latestVersion.files)                                    AS n_files,
       len(list_filter(data.latestVersion.files, f -> f.restricted))    AS n_restricted
FROM bronze_meta WHERE run_id LIKE '%bbbbbb' ORDER BY id;
```

Result

```
id  | n_files | n_restricted
----+---------+-------------
DS1 | 3       | 1
DS2 | NULL    | NULL
DS3 | 1       | 0
(3 rows)
```

SQL

```
-- The same with the Python-style lambda syntax (recent DuckDB versions): identical result.
SELECT data.identifier AS id,
       len(list_filter(data.latestVersion.files, lambda f: f.restricted)) AS n_restricted
FROM bronze_meta WHERE run_id LIKE '%bbbbbb' ORDER BY id;
```

Result

```
id  | n_restricted
----+-------------
DS1 | 1
DS2 | NULL
DS3 | 0
(3 rows)
```

SQL

```sql
-- Transform: a list of file names, total size, and the distinct content types, still one row per dataset.
SELECT data.identifier AS id,
       list_transform(data.latestVersion.files, f -> f.dataFile.filename)                        AS file_names,
       list_sum(list_transform(data.latestVersion.files, f -> f.dataFile.filesize))              AS total_bytes,
       list_sort(list_distinct(list_transform(data.latestVersion.files, f -> f.dataFile.contentType))) AS content_types
FROM bronze_meta WHERE run_id LIKE '%bbbbbb' ORDER BY id;
```

Result

```
id  | file_names                          | total_bytes | content_types
----+-------------------------------------+-------------+--------------------------
DS1 | [data.csv, notes.pdf, codebook.pdf] | 16384       | [application/pdf, text/csv]
DS2 | NULL                                | NULL        | NULL
DS3 | [landing.csv]                       | 51200       | [text/csv]
(3 rows)
```

SQL

```sql
-- Test membership and build text: does the dataset ship a codebook? Names as one comma-separated string.
SELECT data.identifier AS id,
       list_contains(list_transform(data.latestVersion.files, f -> f.dataFile.filename), 'codebook.pdf') AS has_codebook,
       array_to_string(list_transform(data.latestVersion.files, f -> f.dataFile.filename), ', ')         AS file_list
FROM bronze_meta WHERE run_id LIKE '%bbbbbb' ORDER BY id;
```

Result

```
id  | has_codebook | file_list
----+--------------+---------------------------------
DS1 | true         | data.csv, notes.pdf, codebook.pdf
DS2 | NULL         | NULL
DS3 | false        | landing.csv
(3 rows)
```

Python

```python
# Both approaches must agree. The UNNEST way needs a LEFT JOIN, otherwise datasets without files vanish.
lambda_way = con.sql("""
    SELECT data.identifier AS id, run_id,
           COALESCE(len(list_filter(data.latestVersion.files, f -> f.restricted)), 0) AS n
    FROM bronze_meta ORDER BY run_id, id""").fetchall()
unnest_way = con.sql("""
    SELECT data.identifier AS id, run_id, count(*) FILTER (WHERE f.restricted) AS n
    FROM bronze_meta LEFT JOIN UNNEST(data.latestVersion.files) AS t(f) ON true
    GROUP BY data.identifier, run_id ORDER BY run_id, id""").fetchall()
print("same answer:", lambda_way == unnest_way, "| rows:", len(lambda_way))
```

Result

```
same answer: True | rows: 6
```

LESSON 16

Types, time zones and number pitfalls

Level: intermediate | In this project: `parse_ts()` macro, `SET TimeZone = 'UTC'` in `silver.py`, `TIMESTAMP` columns in silver | Needs: nothing

1. Theory

Date and time types:

Type	Holds	Notes
`DATE`	a calendar day	no time, no zone
`TIMESTAMP`	a clock reading, e.g. `2025-05-20 08:30:00`	"naive": says nothing about WHICH moment it is
`TIMESTAMPTZ`	an exact instant (stored as UTC)	shown in the session `TimeZone`; the same instant looks different per zone
`INTERVAL`	a duration (`INTERVAL 30 DAY`)	used in date arithmetic

Conversion rules (the source of most bugs):

- Text WITH a zone (`...Z`, `...+07:00`) can become a `TIMESTAMPTZ` exactly.
- Text WITHOUT a zone cast to `TIMESTAMPTZ` is interpreted in the SESSION time zone: the same text means a different instant on a laptop in Jakarta than on a server in UTC.
- `ts AT TIME ZONE 'UTC'` converts between the two kinds: `TIMESTAMPTZ -> TIMESTAMP` gives the clock reading in UTC; `TIMESTAMP -> TIMESTAMPTZ` says "this clock reading was in UTC".
- Safe habits: store instants in UTC, set the session zone explicitly, and never let the machine's zone decide a result.

Date helpers: `date_trunc('month', x)`, `date_diff('day', a, b)`, `x + INTERVAL 30 DAY`, `strftime(x, '%Y-%m')`, `strptime(text, '%d/%m/%Y')`.

Numbers:

- `DOUBLE` / `FLOAT` are approximate binary numbers: `0.1 + 0.2` is not exactly `0.3`. Use `DECIMAL(p, s)` for money and anything that must add up exactly.
- In DuckDB `/` is true division (`7 / 2 = 3.5`) and `//` is integer division (`7 // 2 = 3`). Other databases differ (PostgreSQL's `7 / 2` is 3 for integers).
- Use `BIGINT` for ids and counts.

2. Real-world case

Dataverse sends `lastUpdateTime` in THREE formats: `2025-02-10 09:00:00` (no zone), `2025-06-01T00:00:00Z` (UTC) and `2025-05-20T08:30:00+07:00` (an offset). The silver table needs one comparable timestamp, because the dedup (lesson 09) orders by it. If the naive text were cast to `TIMESTAMPTZ`, a nightly job on a UTC server and a developer laptop in Jakarta would produce different data, and monthly counts would change near month ends. The project defends itself twice: `silver.py` sets `TimeZone = 'UTC'`, and the `parse_ts()` macro (lesson 17) never lets the session zone decide.

3. Popular usage

- Reporting by day or month (`date_trunc`), cohort and retention analysis, SLA calculations.
- Storing event times from many regions as UTC and converting only for display.
- Money columns as `DECIMAL`, never `DOUBLE`.

4. Example code

SQL

```sql
-- The same TEXT, two session zones: the instants differ by 7 hours (shown here in UTC).
SET TimeZone = 'UTC';
SELECT '2025-02-01 03:00:00'::TIMESTAMPTZ AS instant, ('2025-02-01 03:00:00'::TIMESTAMPTZ) AT TIME ZONE 'UTC' AS as_utc_clock;
```

Result

```
instant                | as_utc_clock
-----------------------+--------------------
2025-02-01 03:00:00+00 | 2025-02-01 03:00:00
(1 row)
```

SQL

```sql
SET TimeZone = 'Asia/Jakarta';
SELECT '2025-02-01 03:00:00'::TIMESTAMPTZ AS instant, ('2025-02-01 03:00:00'::TIMESTAMPTZ) AT TIME ZONE 'UTC' AS as_utc_clock;
```

Result

```
instant                | as_utc_clock
-----------------------+--------------------
2025-02-01 03:00:00+07 | 2025-01-31 20:00:00
(1 row)
```

SQL

```sql
-- Consequence (the session is still Jakarta): the monthly bucket of that one value is JANUARY here,
-- but it would be February on a UTC server. The same data gives different monthly counts.
SELECT date_trunc('month', ('2025-02-01 03:00:00'::TIMESTAMPTZ) AT TIME ZONE 'UTC') AS month_bucket_in_jakarta_session;
```

Result

```
month_bucket_in_jakarta_session
-------------------------------
2025-01-01 00:00:00
(1 row)
```

SQL

```sql
-- Text WITH a zone is unambiguous in any session: both values below are fixed moments in time,
-- and +07:00 is converted to UTC (08:30 at +07:00 is 01:30 UTC).
SELECT x, (x::TIMESTAMPTZ) AT TIME ZONE 'UTC' AS utc_clock
FROM (VALUES ('2025-06-01T00:00:00Z'), ('2025-05-20T08:30:00+07:00')) AS t(x);
```

Result

```
x                         | utc_clock
--------------------------+--------------------
2025-06-01T00:00:00Z      | 2025-06-01 00:00:00
2025-05-20T08:30:00+07:00 | 2025-05-20 01:30:00
(2 rows)
```

SQL

```sql
-- Back to a deterministic session for the rest of the lesson.
SET TimeZone = 'UTC';
SELECT current_setting('TimeZone') AS session_zone;
```

Result

```
session_zone
------------
UTC
(1 row)
```

SQL

```sql
-- Everyday date helpers.
SELECT date_trunc('month', DATE '2025-05-20')                    AS month_start,
       DATE '2025-05-20' + INTERVAL 30 DAY                       AS plus_30_days,
       date_diff('day', DATE '2025-01-01', DATE '2025-05-20')    AS days_between,
       strftime(TIMESTAMP '2025-05-20 08:30:00', '%Y-%m')        AS year_month,
       strptime('20/05/2025', '%d/%m/%Y')                        AS parsed_text;
```

Result

```
month_start         | plus_30_days        | days_between | year_month | parsed_text
--------------------+---------------------+--------------+------------+--------------------
2025-05-01 00:00:00 | 2025-06-19 00:00:00 | 139          | 2025-05    | 2025-05-20 00:00:00
(1 row)
```

SQL

```sql
-- Numbers: DOUBLE is approximate, DECIMAL is exact; / is true division in DuckDB, // is integer division.
SELECT (0.1::DOUBLE + 0.2::DOUBLE) = 0.3::DOUBLE                                AS double_equal,
       (0.1::DECIMAL(10, 2) + 0.2::DECIMAL(10, 2)) = 0.3::DECIMAL(10, 2)        AS decimal_equal,
       7 / 2 AS slash, 7 // 2 AS integer_division;
```

Result

```
double_equal | decimal_equal | slash | integer_division
-------------+---------------+-------+-----------------
false        | true          | 3.5   | 3
(1 row)
```

LESSON 17

Macros: reusable SQL expressions

Level: intermediate | In this project: `CREATE OR REPLACE TEMP MACRO parse_ts(x)` | Needs: lessons 16, 18

1. Theory

A **macro** gives a name to a SQL expression (scalar macro) or to a whole query (table macro). At query time the engine substitutes the macro body, so there is no function-call overhead and the optimizer sees through it.

```
CREATE MACRO name(a, b, c := default) AS expression;          -- scalar macro
CREATE MACRO name(a) AS TABLE SELECT ... WHERE col = a;       -- table macro, used as: FROM name(x)
```

- `CREATE OR REPLACE` makes re-runs safe; `TEMP` limits it to the session (nothing is stored in a database file).
- Parameters can have defaults and be passed by name (`digits := 3`).
- A parameter name hides a column of the same name inside the body, so choose parameter names that no column uses.

Alternatives and when to prefer them:

- **View**: a saved query without parameters. Use it for a fixed dataset.
- **Python UDF** (`con.create_function`): needs logic SQL cannot express, but runs row by row in Python: much slower.
- **Copy-paste**: the macro's enemy. Three copies of the same date logic WILL diverge.

2. Real-world case

Three silver columns (`create_time`, `last_update_time`, `release_time`) all need the same tricky parsing: timestamps with a zone are converted to UTC, timestamps without one keep their clock time, junk becomes NULL (lesson 16). Writing that `CASE` three times, plus once more in the quality check that counts unparseable values, would invite four slightly different versions. The macro `parse_ts(x)` is defined once and every use is one short call. The quality check calls the SAME macro, so "what counts as valid" cannot drift apart from "what was parsed".

3. Popular usage

- Shared business rules: "is this a valid email", "normalize country code", "fiscal quarter of a date".
- dbt macros (Jinja, a different mechanism) serve the same purpose across a whole warehouse.
- Parameterized reports as table macros.

4. Example code

SQL

```sql
-- The project's macro: zone-aware text -> UTC; no zone -> keep the clock time; invalid -> NULL.
CREATE OR REPLACE TEMP MACRO parse_ts(x) AS
    CASE
        WHEN x IS NULL THEN NULL
        WHEN regexp_matches(x, '(Z|[+-][0-9]{2}:?[0-9]{2})$')
            THEN TRY_CAST(x AS TIMESTAMPTZ) AT TIME ZONE 'UTC'
        ELSE TRY_CAST(x AS TIMESTAMP)
    END;
SELECT x AS raw_text, parse_ts(x) AS parsed
FROM (VALUES ('2025-02-10 09:00:00'), ('2025-06-01T00:00:00Z'), ('2025-05-20T08:30:00+07:00'),
             ('garbage'), (NULL)) AS t(x);
```

Result

```
raw_text                  | parsed
--------------------------+--------------------
2025-02-10 09:00:00       | 2025-02-10 09:00:00
2025-06-01T00:00:00Z      | 2025-06-01 00:00:00
2025-05-20T08:30:00+07:00 | 2025-05-20 01:30:00
garbage                   | NULL
NULL                      | NULL
(5 rows)
```

SQL

```sql
-- The macro does not depend on the session zone: the same query in a Jakarta session gives the same answer.
SET TimeZone = 'Asia/Jakarta';
SELECT x AS raw_text, parse_ts(x) AS parsed
FROM (VALUES ('2025-02-10 09:00:00'), ('2025-06-01T00:00:00Z'), ('2025-05-20T08:30:00+07:00')) AS t(x);
```

Result

```
raw_text                  | parsed
--------------------------+--------------------
2025-02-10 09:00:00       | 2025-02-10 09:00:00
2025-06-01T00:00:00Z      | 2025-06-01 00:00:00
2025-05-20T08:30:00+07:00 | 2025-05-20 01:30:00
(3 rows)
```

SQL

```sql
SET TimeZone = 'UTC';
-- A scalar macro with a default parameter and a safe division.
CREATE OR REPLACE MACRO pct(part, whole, digits := 1) AS round(100.0 * part / NULLIF(whole, 0), digits);
SELECT pct(1, 3) AS default_digits, pct(1, 3, digits := 3) AS named_digits, pct(1, 0) AS zero_whole;
```

Result

```
default_digits | named_digits | zero_whole
---------------+--------------+-----------
33.3           | 33.333       | NULL
(1 row)
```

Python

```python
run_sql_file("setup_bronze.sql")
```

SQL

```sql
-- A table macro: a parameterized query used like a table (files of one dataset in the newest run).
CREATE OR REPLACE MACRO files_of(dataset_ident) AS TABLE
    SELECT data.identifier AS id, f.dataFile.filename AS filename, f.restricted AS restricted
    FROM bronze_meta, UNNEST(data.latestVersion.files) AS t(f)
    WHERE data.identifier = dataset_ident AND run_id LIKE '%bbbbbb';
SELECT * FROM files_of('DS1') ORDER BY filename;
```

Result

```
id  | filename     | restricted
----+--------------+-----------
DS1 | codebook.pdf | false
DS1 | data.csv     | false
DS1 | notes.pdf    | true
(3 rows)
```

SQL

```sql
-- Macros compose: use parse_ts inside a query like any function, here to count the unparseable values
-- (this is the quality-check pattern: it uses the SAME macro as the parsing).
SELECT count(*) FILTER (WHERE data.latestVersion.lastUpdateTime IS NOT NULL
                        AND parse_ts(data.latestVersion.lastUpdateTime) IS NULL) AS unparseable_last_update,
       count(*) FILTER (WHERE parse_ts(data.latestVersion.lastUpdateTime) IS NOT NULL)  AS parsed_ok
FROM bronze_meta;
```

Result

```
unparseable_last_update | parsed_ok
------------------------+----------
0                       | 4
(1 row)
```

LESSON 18

Safe casting: CAST, TRY_CAST and validating raw values

Level: intermediate | In this project: `TRY_CAST(publication_date_raw AS DATE)` and the `_raw` columns in `stg_datasets_all` | Needs: lesson 03

1. Theory

Converting text to dates and numbers is where dirty data bites.

- `CAST(x AS type)` raises an error on the first value that does not fit, and the whole query fails.
- `TRY_CAST(x AS type)` returns NULL for values that do not fit. The query completes.
- `try(expr)` (DuckDB) wraps ANY expression: if it raises an error, the result is NULL.

Neither is "better": choose by what you want to happen to a bad value.

- Strict stage (you want to be stopped): `CAST`.
- Reporting stage (you want to survive and count): `TRY_CAST`.

The raw / typed twin pattern. Never throw the original text away. Keep the text (`publication_date_raw`) AND the typed column (`publication_date`). Then "bad values" are exactly the rows where raw is NOT NULL and typed IS NULL, and a quality check can count them. If you only keep the typed column, a NULL could mean "missing at the source" or "failed to parse" and you cannot tell which.

Casting also has silent surprises: decimal text cast to an integer type is ROUNDED (`'12.5'` becomes 13) instead of failing. Cast to `DECIMAL(p, s)` when the exact value matters.

Validation can be layered: a regular expression for the format (`^\d{4}-\d{2}-\d{2}$`), then `TRY_CAST` for real validity (the format of `2025-13-45` is fine, but there is no month 13).

2. Real-world case

The sample run contains a dataset (DS3) whose `publicationDate` is `2025-13-45`. A plain `CAST` would have stopped the whole silver build for one bad record. The project uses `TRY_CAST`, keeps `publication_date_raw` in the staging table, and a quality check (severity "warn") reports `publication_date text that is not a valid date: 1`. The build still publishes, and the data owner gets a precise list of what to fix at the source.

3. Popular usage

- Loading CSV and JSON where numbers arrive as text with stray characters.
- Data quality dashboards: "N values failed to parse in column X".
- Staging layers in dbt: `safe_cast` / `try_cast` macros.

4. Example code

Python

```
run_sql_file("setup_bronze.sql")
```

SQL

```sql
-- Raw text next to the typed value. DS3's date is invalid: the typed column becomes NULL, the raw text is kept.
SELECT data.identifier AS id,
       data.publicationDate                       AS publication_date_raw,
       TRY_CAST(data.publicationDate AS DATE)     AS publication_date
FROM bronze_meta WHERE run_id LIKE '%bbbbbb' ORDER BY id;
```

Result

```
id  | publication_date_raw | publication_date
----+----------------------+-----------------
DS1 | 2025-03-01           | 2025-03-01
DS2 | 2024-06-01           | 2024-06-01
DS3 | 2025-13-45           | NULL
(3 rows)
```

SQL

```sql
-- The quality check: raw present but typed NULL. Missing (NULL raw) is NOT counted as bad.
SELECT count(*) FILTER (WHERE data.publicationDate IS NOT NULL
                         AND TRY_CAST(data.publicationDate AS DATE) IS NULL)  AS bad_dates,
       count(*) FILTER (WHERE data.publicationDate IS NULL)                   AS missing_dates
FROM bronze_meta;
```

Result

```
bad_dates | missing_dates
----------+--------------
1         | 1
(1 row)
```

Python

```python
# CAST stops at the first bad value; TRY_CAST and try() turn it into NULL.
try:
    con.execute("SELECT CAST('2025-13-45' AS DATE)")
except duckdb.Error as e:
    print("CAST      ->", str(e).splitlines()[0])
print("TRY_CAST  ->", con.sql("SELECT TRY_CAST('2025-13-45' AS DATE)").fetchone()[0])
print("try()     ->", con.sql("SELECT try(strptime('13/45/2025', '%m/%d/%Y'))").fetchone()[0])
```

Result

```
CAST      -> Conversion Error: date field value out of range: "2025-13-45"
TRY_CAST  -> None
try()     -> None
```

SQL

```sql
-- Layered validation: the format check passes for 2025-13-45, only the real cast catches it.
SELECT x,
       regexp_matches(x, '^\d{4}-\d{2}-\d{2}$')                    AS format_ok,
       TRY_CAST(x AS DATE) IS NOT NULL                              AS really_a_date,
       CASE WHEN x IS NULL                                THEN 'missing'
            WHEN TRY_CAST(x AS DATE) IS NULL              THEN 'unparseable'
            ELSE 'ok' END                                           AS status
FROM (VALUES ('2025-03-01'), ('2025-13-45'), ('March 2025'), (NULL)) AS t(x);
```

Result

```
x          | format_ok | really_a_date | status
-----------+-----------+---------------+------------
2025-03-01 | true      | true          | ok
2025-13-45 | true      | false         | unparseable
March 2025 | false     | false         | unparseable
NULL       | NULL      | false         | missing
(4 rows)
```

SQL

```sql
-- Numbers: junk and '' become NULL. Careful: '12.5' as INTEGER is silently ROUNDED to 13, not rejected.
-- If the exact value matters, cast to DECIMAL.
SELECT x,
       TRY_CAST(x AS INTEGER)         AS as_integer,
       TRY_CAST(x AS DECIMAL(10, 2))  AS as_decimal
FROM (VALUES ('12'), ('12.5'), ('abc'), ('')) AS t(x);
```

Result

```
x    | as_integer | as_decimal
-----+------------+-----------
12   | 12         | 12.00
12.5 | 13         | 12.50
abc  | NULL       | NULL
     | NULL       | NULL
(4 rows)
```

LESSON 19

Tables, views, temp tables and idempotent rebuilds

Level: intermediate | In this project: `CREATE OR REPLACE TEMP TABLE`, full rebuild per build id, `_SUCCESS` commit marker | Needs: nothing

1. Theory

Ways to store a query result:

Object	Stores	Lives	Use for
`CREATE TABLE t AS SELECT ...` (CTAS)	the rows	until dropped	results other jobs read
`CREATE TEMP TABLE`	the rows	this connection only	working steps inside one job
`CREATE VIEW v AS SELECT ...`	only the query (re-run on every read)	until dropped	a named, always-fresh query
`CREATE OR REPLACE ...`	same as above, replacing any old one		re-runnable scripts

Idempotent means: running the job twice (or ten times) leaves the SAME result as running it once. Jobs get retried (a crash at 03:00, a scheduler re-run), so a job that is not idempotent eventually corrupts data.

- NOT idempotent: a plain `INSERT` of the day's rows (a retry duplicates them).
- Idempotent: `CREATE OR REPLACE TABLE ... AS`, "delete the partition then insert it again", upsert (lesson 21).

Transactions make several statements all-or-nothing: `BEGIN; ...; COMMIT;` (or `ROLLBACK;` to undo). A common safe pattern is the **atomic swap**: build `report_new` next to the live `report`, then in one transaction drop `report` and rename `report_new`, so readers never see a half-built table.

Files have no transactions. A pipeline that writes many files cannot be atomic, so it uses a **commit marker**: write all data files first, then the manifest, then an empty `_SUCCESS` file last. Readers only trust folders that have it.

2. Real-world case

The silver build always rebuilds everything from the raw layer under a fresh `build=<id>` folder, with `CREATE OR REPLACE` for every working table. A retry cannot duplicate anything. And because upload of ~7 files is not atomic, `_SUCCESS` is written last: a build that died half way has no `_SUCCESS`, so readers ignore it. The same reasoning appears in the raw layer, where a run only gets `_SUCCESS` if EVERY dataverse succeeded.

3. Popular usage

- dbt materializations: `view`, `table`, `incremental`: the same menu as above.
- Airflow / cron retries: tasks must be safe to run again.
- Blue/green table swaps for dashboards that must never show a half-loaded table.

4. Example code

SQL

```sql
CREATE TABLE catalogue_log (run_id VARCHAR, n_datasets INTEGER);
```

Python

```python
# A NON-idempotent job: it appends. The scheduler retries it after a failure, so the run is stored twice.
def nightly_job_with_insert():
    con.execute("INSERT INTO catalogue_log VALUES ('run_B', 4)")

nightly_job_with_insert()
nightly_job_with_insert()
print("append twice  ->", con.sql("SELECT run_id, count(*) AS copies FROM catalogue_log GROUP BY run_id").fetchall())
```

Result

```
append twice  -> [('run_B', 2)]
```

Python

```python
# An idempotent job: it replaces. Run it twice, get the same single row.
def nightly_job_with_replace():
    con.execute("CREATE OR REPLACE TABLE catalogue_snapshot AS SELECT 'run_B' AS run_id, 4 AS n_datasets")

nightly_job_with_replace()
nightly_job_with_replace()
print("replace twice ->", con.sql("SELECT * FROM catalogue_snapshot").fetchall())
```

Result

```
replace twice -> [('run_B', 4)]
```

SQL

```sql
-- Idempotent append: delete the run's rows, insert them again, inside one transaction.
BEGIN;
DELETE FROM catalogue_log WHERE run_id = 'run_B';
INSERT INTO catalogue_log VALUES ('run_B', 4);
COMMIT;
SELECT run_id, count(*) AS copies FROM catalogue_log GROUP BY run_id;
```

Result

```
run_id | copies
-------+-------
run_B  | 1
(1 row)
```

Python

```python
# ROLLBACK undoes everything since BEGIN: all-or-nothing.
con.execute("BEGIN")
con.execute("INSERT INTO catalogue_log VALUES ('run_C', 99)")
print("inside the transaction:", con.sql("SELECT count(*) FROM catalogue_log WHERE run_id = 'run_C'").fetchone()[0])
con.execute("ROLLBACK")
print("after ROLLBACK        :", con.sql("SELECT count(*) FROM catalogue_log WHERE run_id = 'run_C'").fetchone()[0])
```

Result

```
inside the transaction: 1
after ROLLBACK        : 0
```

SQL

```sql
-- A view re-runs its query every time; a table is a snapshot taken when it was created.
CREATE TABLE src (x INTEGER);
INSERT INTO src VALUES (1), (2);
CREATE VIEW v_src AS SELECT sum(x) AS total FROM src;
CREATE TABLE t_src AS SELECT sum(x) AS total FROM src;
INSERT INTO src VALUES (10);
SELECT (SELECT total FROM v_src) AS view_total, (SELECT total FROM t_src) AS table_total;
```

Result

```
view_total | table_total
-----------+------------
13         | 3
(1 row)
```

Python

```python
# Atomic swap: build the new table next to the live one, then swap names inside one transaction.
con.execute("CREATE TABLE report AS SELECT 'old data' AS note")
con.execute("BEGIN")
con.execute("CREATE TABLE report_new AS SELECT 'new data' AS note")
con.execute("DROP TABLE report")
con.execute("ALTER TABLE report_new RENAME TO report")
con.execute("COMMIT")
print("after the swap:", con.sql("SELECT note FROM report").fetchone()[0])
```

Result

```
after the swap: new data
```

SQL

```sql
-- What did we create? "temporary" is true for TEMP tables (only visible to this connection).
CREATE TEMP TABLE scratch AS SELECT 1 AS x;
SELECT table_name, temporary FROM duckdb_tables() ORDER BY table_name;
```

Result

```
table_name         | temporary
-------------------+----------
catalogue_log      | false
catalogue_snapshot | false
report             | false
scratch            | true
src                | false
t_src              | false
(6 rows)
```

LESSON 20

Medallion layers and keeping history (SCD type 1 and 2)

Level: advanced | In this project: bronze (raw bucket) -> silver (`silver_*`), `silver_datasets` keeps the current version only | Needs: lessons 08, 09

1. Theory

Medallion architecture organizes data by quality:

- **Bronze**: raw, append-only, exactly as received. Never edited. The source of truth you can always rebuild from.
- **Silver**: cleaned, typed, de-duplicated, conformed. One row per real-world thing, ready to join.
- **Gold**: business-level tables for one use case: aggregates, star schemas (fact + dimension tables), feature tables.

Slowly changing dimensions (SCD) are about what happens when an attribute changes:

- **Type 1**: overwrite. Only the current value exists. Simple and small. (`silver_datasets` is type 1.)
- **Type 2**: keep every version with a validity range: `valid_from`, `valid_to`, and often an `is_current` flag. Lets you answer "what was it on date X?". Built with `lag()` to detect a change and `lead()` to close the range.
- Type 3 keeps just the previous value in an extra column. Rarely used.

Type 2 recipe:

1. Take the snapshots over time (here: one per extraction run).
2. `lag(attr) OVER (PARTITION BY key ORDER BY snapshot_date)` gives the previous value.
3. Keep only the rows where `attr IS DISTINCT FROM prev` (a change, or the first sighting).
4. `valid_to = lead(snapshot_date) OVER (PARTITION BY key ORDER BY snapshot_date)` ; NULL means "still current".
5. Point-in-time query: `WHERE d >= valid_from AND (valid_to IS NULL OR d < valid_to)` .

Limitation: a key that simply stops appearing is not "changed", so a naive type 2 keeps showing it as current. Handle deletes explicitly (a `deleted_at` flag, or compare with the latest snapshot).

2. Real-world case

Silver keeps the NEWEST title of each dataset, which is what most users want. A data steward then asks: "What was the title of DS1 when the 12 October report was produced?" Silver cannot answer, because it overwrote the old title. But bronze still has every run, so a history table can be built from it without touching the extraction. This is the benefit of keeping bronze immutable.

3. Popular usage

- Customer / product / employee dimensions in a warehouse (addresses, prices and job titles change).
- Audit trails and "as of" reporting in finance and compliance.
- Gold-layer star schemas: a fact table joins to the dimension row that was valid at the event date.

4. Example code

Python

```
build_silver()   # runs the project's real flatten_datasets_metadata.sql on the sample files
```

SQL

```
-- The layers by size: bronze has one row per record per run, silver one row per dataset.
SELECT 'bronze_meta           (1 row per record per run)' AS layer_table, count(*) AS n FROM bronze_meta
UNION ALL SELECT 'silver_datasets        (1 row per dataset)',            count(*) FROM silver_datasets
UNION ALL SELECT 'silver_dataset_authors (1 row per dataset+author)',     count(*) FROM silver_dataset_authors
UNION ALL SELECT 'silver_dataset_subjects(1 row per dataset+subject)',    count(*) FROM silver_dataset_subjects;
```

Result

```
layer_table                                        | n
---------------------------------------------------+--
bronze_meta           (1 row per record per run)   | 6
silver_datasets        (1 row per dataset)         | 4
silver_dataset_authors (1 row per dataset+author)  | 4
silver_dataset_subjects(1 row per dataset+subject) | 5
(4 rows)
```

SQL

```
-- SCD type 1: only the current title exists. DS1 shows the run-B title; the old one is gone from silver.
SELECT global_id, source_run_id, title FROM silver_datasets ORDER BY global_id;
```

Result

```
global_id        | source_run_id           | title
-----------------+-------------------------+----------------------
doi:10.99999/DS1 | 20261013T000000Z_bbbbbb | Rice yield survey 2025
doi:10.99999/DS2 | 20261013T000000Z_bbbbbb | NULL
doi:10.99999/DS3 | 20261013T000000Z_bbbbbb | Fish landing records
doi:10.99999/DS4 | 20261012T000000Z_aaaaaa | Soil moisture sensors
(4 rows)
```

SQL

```sql
-- SCD type 2 built from bronze: one row per distinct title, with the date range it was valid.
CREATE TEMP TABLE dataset_history AS
WITH seen AS (
    SELECT f.global_id, r.extract_date, f.value ->> '$' AS title
    FROM stg_fields f
    JOIN (SELECT DISTINCT run_id, extract_date FROM bronze_meta) r USING (run_id)
    WHERE f.type_name = 'title'
),
changes AS (
    SELECT *, lag(title) OVER (PARTITION BY global_id ORDER BY extract_date) AS prev_title FROM seen
),
versions AS (SELECT * FROM changes WHERE title IS DISTINCT FROM prev_title)
SELECT global_id, title, extract_date AS valid_from,
       lead(extract_date) OVER (PARTITION BY global_id ORDER BY extract_date) AS valid_to
FROM versions;

SELECT *, valid_to IS NULL AS is_current FROM dataset_history ORDER BY global_id, valid_from;
```

Result

```
global_id        | title                  | valid_from | valid_to   | is_current
-----------------+------------------------+------------+------------+-----------
doi:10.99999/DS1 | Rice yield survey      | 2026-10-12 | 2026-10-13 | false
doi:10.99999/DS1 | Rice yield survey 2025 | 2026-10-13 | NULL       | true
doi:10.99999/DS3 | Fish landing records   | 2026-10-13 | NULL       | true
doi:10.99999/DS4 | Soil moisture sensors  | 2026-10-12 | NULL       | true
(4 rows)
```

SQL

```sql
-- Point-in-time: the catalogue as it looked on 12 October.
SELECT global_id, title FROM dataset_history
WHERE DATE '2026-10-12' >= valid_from AND (valid_to IS NULL OR DATE '2026-10-12' < valid_to)
ORDER BY global_id;
```

Result

```
global_id        | title
-----------------+----------------------
doi:10.99999/DS1 | Rice yield survey
doi:10.99999/DS4 | Soil moisture sensors
(2 rows)
```

SQL

```sql
-- ...and on 13 October. DS1 now has its new title. But DS4 is listed although it was NOT in the 13 October run.
SELECT global_id, title FROM dataset_history
WHERE DATE '2026-10-13' >= valid_from AND (valid_to IS NULL OR DATE '2026-10-13' < valid_to)
ORDER BY global_id;
```

Result

```
global_id        | title
-----------------+------------------------
doi:10.99999/DS1 | Rice yield survey 2025
doi:10.99999/DS3 | Fish landing records
doi:10.99999/DS4 | Soil moisture sensors
(3 rows)
```

SQL

```sql
-- The limitation made visible: "current" in the history but absent from the newest run (a deleted or hidden dataset).
SELECT h.global_id, h.title
FROM dataset_history h
WHERE h.valid_to IS NULL
  AND NOT EXISTS (SELECT 1 FROM stg_fields f WHERE f.global_id = h.global_id AND f.run_id LIKE '%bbbbbb');
```

Result

```
global_id        | title
-----------------+----------------------
doi:10.99999/DS4 | Soil moisture sensors
(1 row)
```

LESSON 21

Incremental loads and upserts (MERGE, ON CONFLICT)

Level: advanced

In this project: not used yet (silver is a full rebuild); needed when loading to PostgreSQL (`pg_*` settings in `config.py`)

Needs: lessons 03, 19

1. Theory

Full refresh recomputes everything each time: simple, always correct, but cost grows with the data (the silver `--runs all` mode re-reads every raw run). **Incremental** processes only what is new or changed, using a **watermark**: a value that says "everything up to here is already done", typically `max(last_update_time)` of the target.

Writing the changes into a table that already has rows is an **upsert**: update the row if the key exists, insert it if not.

- `INSERT ... ON CONFLICT (key) DO UPDATE SET col = excluded.col [WHERE condition]` (PostgreSQL, DuckDB, SQLite). Needs a PRIMARY KEY or UNIQUE constraint on `key`. `excluded` is the row you tried to insert.
- `MERGE INTO target USING source ON condition WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...` (SQL standard; PostgreSQL 15+, DuckDB, SQL Server, Snowflake, BigQuery). More flexible: it can also `DELETE`.

Rules for safe incremental loads:

1. A reliable key and a reliable "version" column (last update time).
2. The upsert must be idempotent: loading the same batch twice changes nothing.
3. Guard against **stale** data: only overwrite when the incoming row is newer (`WHERE excluded.last_update > target.last_update`).
4. Decide how **deletes** are signalled: the source may just stop returning a row. Use a soft-delete flag or a periodic full refresh.
5. Keep an occasional full rebuild as the safety net.

2. Real-world case

The silver stage is a full rebuild, which is fine at 370 datasets. The planned next stage loads gold data into PostgreSQL (see the `pg_*` settings), where dropping and reloading a table is not acceptable if dashboards are reading it. There the load must be an idempotent upsert: a changed dataset replaces its row, a new dataset is inserted, and a stale row from an old run must NOT overwrite a newer one.

3. Popular usage

- Change data capture (CDC) feeds applied to a warehouse table with MERGE.
- dbt `incremental` models (`unique_key` + `is_incremental()` watermark filter).
- Syncing a CRM or catalogue where only a small share of rows change daily.

4. Example code

SQL

```sql
-- The target table. The PRIMARY KEY is what ON CONFLICT needs.
CREATE TABLE silver_ds (global_id VARCHAR PRIMARY KEY, title VARCHAR, last_update TIMESTAMP);
INSERT INTO silver_ds VALUES
    ('DS1', 'Rice yield survey',      TIMESTAMP '2025-01-01 00:00:00'),
    ('DS4', 'Soil moisture sensors',  TIMESTAMP '2025-02-10 09:00:00');
```

SQL

```sql
-- The next batch: DS1 changed, DS3 is new, DS4 arrives with an OLDER timestamp (a stale re-delivery).
CREATE TABLE incoming AS
SELECT * FROM (VALUES
    ('DS1', 'Rice yield survey 2025', TIMESTAMP '2025-06-01 00:00:00'),
    ('DS3', 'Fish landing records',   TIMESTAMP '2025-05-20 01:30:00'),
    ('DS4', 'OLD STALE TITLE',        TIMESTAMP '2025-01-01 00:00:00')
) AS t(global_id, title, last_update);
```

SQL

```sql
-- The watermark: what is the newest thing we already have? Only rows newer than this need to be fetched.
SELECT (SELECT max(last_update) FROM silver_ds) AS watermark,
       (SELECT count(*) FROM incoming WHERE last_update > (SELECT max(last_update) FROM silver_ds)) AS rows_worth_processing;
```

Result

```
watermark           | rows_worth_processing
--------------------+----------------------
2025-02-10 09:00:00 | 2
(1 row)
```

SQL

```sql
-- Upsert with a stale-data guard: DS1 updated, DS3 inserted, DS4 untouched because the incoming row is older.
INSERT INTO silver_ds
SELECT * FROM incoming
ON CONFLICT (global_id) DO UPDATE
    SET title = excluded.title, last_update = excluded.last_update
    WHERE excluded.last_update > silver_ds.last_update;
SELECT * FROM silver_ds ORDER BY global_id;
```

Result

```
global_id | title                  | last_update
----------+------------------------+--------------------
DS1       | Rice yield survey 2025 | 2025-06-01 00:00:00
DS3       | Fish landing records   | 2025-05-20 01:30:00
DS4       | Soil moisture sensors  | 2025-02-10 09:00:00
(3 rows)
```

Python

```python
# Idempotency test: apply the SAME batch again. Nothing may change.
upsert = """INSERT INTO silver_ds SELECT * FROM incoming
            ON CONFLICT (global_id) DO UPDATE SET title = excluded.title, last_update = excluded.last_update
            WHERE excluded.last_update > silver_ds.last_update"""
before = con.sql("SELECT * FROM silver_ds ORDER BY global_id").fetchall()
con.execute(upsert)
after = con.sql("SELECT * FROM silver_ds ORDER BY global_id").fetchall()
print("second run changed nothing:", before == after)
```

Result

```
second run changed nothing: True
```

SQL

```sql
-- The same logic with MERGE (it can also delete). A copy of the original target is merged with the same batch.
CREATE TABLE silver_ds2 AS
SELECT * FROM (VALUES
    ('DS1', 'Rice yield survey',     TIMESTAMP '2025-01-01 00:00:00'),
    ('DS4', 'Soil moisture sensors', TIMESTAMP '2025-02-10 09:00:00')
) AS t(global_id, title, last_update);

MERGE INTO silver_ds2 AS t
USING incoming AS s ON t.global_id = s.global_id
WHEN MATCHED AND s.last_update > t.last_update THEN UPDATE SET title = s.title, last_update = s.last_update
WHEN NOT MATCHED THEN INSERT VALUES (s.global_id, s.title, s.last_update);

SELECT * FROM silver_ds2 ORDER BY global_id;
```

Result

```
global_id | title                  | last_update
----------+------------------------+--------------------
DS1       | Rice yield survey 2025 | 2025-06-01 00:00:00
DS3       | Fish landing records   | 2025-05-20 01:30:00
DS4       | Soil moisture sensors  | 2025-02-10 09:00:00
(3 rows)
```

PostgreSQL (shown, not run)

```sql
-- PostgreSQL: the same two statements. ON CONFLICT works as in DuckDB; MERGE needs PostgreSQL 15 or newer. (Not run here.)
INSERT INTO silver.datasets (global_id, title, last_update)
SELECT global_id, title, last_update FROM staging.incoming
ON CONFLICT (global_id) DO UPDATE
    SET title = EXCLUDED.title, last_update = EXCLUDED.last_update
    WHERE EXCLUDED.last_update > silver.datasets.last_update;
```

LESSON 22

Data quality as SQL: checks, severity and gates

Level: advanced | In this project: `qa_checks` (section 6 of the flatten SQL) and `check_quality()` in `silver.py` | Needs: lessons 02, 03, 18

1. Theory

A data quality check is a query that COUNTS BAD ROWS. Zero means pass. That one convention makes checks easy to stack (`UNION ALL`), report and automate. Dimensions to cover:

Dimension	Question	Typical SQL
Uniqueness	is the key unique?	`GROUP BY key HAVING count(*) > 1`
Completeness	are required values present?	`WHERE col IS NULL`
Validity	does the value parse / fit the rule?	raw NOT NULL and `TRY_CAST` IS NULL; `NOT IN (accepted list)`
Referential integrity	does every child have a parent?	anti join child to parent
Reconciliation	do counts match between layers and the manifest?	compare `count(*)` of two tables
Consistency	do related values agree?	one entity under two parents; `end < start`
Freshness	is the data recent enough?	`max(updated_at)` vs the reference date

Severity separates two different situations:

- `error`: the OUTPUT is wrong (duplicate keys, orphans, missing rows). A **gate** stops the build and nothing is published.
- `warn`: the SOURCE data is messy (an invalid date typed by a depositor). Log it, store it in the manifest, still publish, and send it to the data owner.

Good checks are **debuggable**: besides the count, you can list the failing keys. And they run **before** publishing, not after users have seen the data.

2. Real-world case

`silver.py` publishes Parquet files that other people will read. Two gates run before any upload: (1) the number of rows DuckDB loaded must equal the number the raw manifests promise (catches a truncated file), (2) every `error` check must be

1. The sample data contains one real source problem: DS3's publication date `2025-13-45`. It must NOT block the build (it is

the depositor's mistake), but it must be reported with the dataset id so someone can fix it. Hence severity `warn`.

3. Popular usage

- dbt tests (`unique`, `not_null`, `relationships`, `accepted_values`) and Great Expectations / Soda suites are this same idea with more tooling.

- Row-count reconciliation between source system and warehouse after every load.
- Freshness alerts ("no new rows since yesterday").

4. Example code

Python

```
build_silver()   # runs the project's real flatten SQL, including its 8 qa_checks, on the sample data
```

SQL

```
-- The project's own checks: all 0 except one warning (DS3's invalid publication date).
SELECT check_name, severity, bad_rows FROM qa_checks ORDER BY severity, check_name;
```

Result

```
check_name                                                      | severity | bad_rows
----------------------------------------------------------------+----------+---------
author row without a parent dataset (orphan)                    | error    | 0
bronze datasets missing from silver (distinct global_id count... | error    | 0
dataset without a title                                         | error    | 0
duplicate global_id in silver_datasets                          | error    | 0
dataset found under more than one dataverse (dataverse choice... | warn     | 0
dataset without any author                                      | warn     | 0
last_update_time text that is not a valid timestamp             | warn     | 0
publication_date text that is not a valid date                  | warn     | 1
(8 rows)
```

SQL

```
-- Debuggable: not just "1 bad row" but WHICH row, and the raw text that failed.
SELECT global_id, publication_date_raw
FROM stg_datasets_all
WHERE publication_date_raw IS NOT NULL AND TRY_CAST(publication_date_raw AS DATE) IS NULL;
```

Result

```
global_id        | publication_date_raw
-----------------+---------------------
doi:10.99999/DS3 | 2025-13-45
(1 row)
```

SQL

```
-- Writing more checks: accepted values (error), a range rule (warn), and freshness against a reference date (warn).
SELECT 'version_state outside the accepted list' AS check_name, 'error' AS severity, count(*) AS bad_rows
FROM silver_datasets
WHERE version_state IS NOT NULL AND version_state NOT IN ('RELEASED', 'DRAFT', 'DEACCESSIONED')
UNION ALL
SELECT 'publication_date later than the reference date', 'warn', count(*)
FROM silver_datasets WHERE publication_date > DATE '2026-10-13'
UNION ALL
SELECT 'newest update is older than 365 days (stale source)', 'warn',
       CASE WHEN max(last_update_time) < TIMESTAMP '2026-10-13 00:00:00' - INTERVAL 365 DAY THEN 1 ELSE 0 END
FROM silver_datasets;
```

Result

```
check_name                                          | severity | bad_rows
----------------------------------------------------+----------+---------
version_state outside the accepted list             | error    | 0
publication_date later than the reference date      | warn     | 0
newest update is older than 365 days (stale source) | warn     | 1
(3 rows)
```

Python

```python
# The gate, in the same shape as silver.check_quality(): reconcile counts, then fail on any error-severity check.
class QualityGateError(Exception):
    pass

def gate(expected_rows):
    loaded = con.sql("SELECT count(*) FROM bronze_meta").fetchone()[0]
    if loaded != expected_rows:
        raise QualityGateError(f"loaded {loaded} rows but the manifests promise {expected_rows}")
    checks = con.sql("SELECT check_name, severity, bad_rows FROM qa_checks ORDER BY severity, check_name").fetchall()
    for name, severity, bad in checks:
        status = "ok  " if bad == 0 else ("WARN" if severity == "warn" else "FAIL")
        print(f"  {status} {name}" + (f" ({bad})" if bad else ""))
    failed = [name for name, severity, bad in checks if severity == "error" and bad > 0]
    if failed:
        raise QualityGateError("quality gate failed, nothing published: " + "; ".join(failed))

print("1) normal data:")
gate(expected_rows=6)
print("2) the manifest says 7 rows but only 6 were loaded:")
try:
    gate(expected_rows=7)
except QualityGateError as e:
    print("  stopped ->", e)
print("3) an error-severity check finds bad rows:")
con.execute("INSERT INTO qa_checks VALUES ('simulated duplicate keys', 'error', 3)")
try:
    gate(expected_rows=6)
except QualityGateError as e:
    print("  stopped ->", e)
```

Result

```
1) normal data:
  ok   author row without a parent dataset (orphan)
  ok   bronze datasets missing from silver (distinct global_id count differs)
  ok   dataset without a title
  ok   duplicate global_id in silver_datasets
  ok   dataset found under more than one dataverse (dataverse choice is arbitrary)
  ok   dataset without any author
  ok   last_update_time text that is not a valid timestamp
  WARN publication_date text that is not a valid date (1)
2) the manifest says 7 rows but only 6 were loaded:
  stopped -> loaded 6 rows but the manifests promise 7
3) an error-severity check finds bad rows:
  ok   author row without a parent dataset (orphan)
  ok   bronze datasets missing from silver (distinct global_id count differs)
  ok   dataset without a title
  ok   duplicate global_id in silver_datasets
  FAIL simulated duplicate keys (3)
  ok   dataset found under more than one dataverse (dataverse choice is arbitrary)
  ok   dataset without any author
  ok   last_update_time text that is not a valid timestamp
  WARN publication_date text that is not a valid date (1)
  stopped -> quality gate failed, nothing published: simulated duplicate keys
```

LESSON 23

Performance and storage: Parquet, pushdown and partitioning

Level: advanced

In this project: `COPY ... (FORMAT parquet, COMPRESSION zstd)`, `extract_date=/run_id=` folders, `--runs latest`

Needs: lesson 10

1. Theory

Row stores versus column stores. A transactional database (PostgreSQL) keeps whole rows together: good for "read or update one record". An analytic engine (DuckDB, Parquet, BigQuery) keeps each COLUMN together: a query that needs 2 of 40 columns reads about 2/40 of the data, and similar values next to each other compress very well.

Parquet is the standard columnar file format. Inside a file: rows are grouped into **row groups**; each column chunk has min/max **statistics**; values are encoded and compressed (zstd here). Types are stored, so no re-parsing or guessing.

Pushdown: the engine pushes work down into the file scan.

- *Projection pushdown*: read only the columns the query uses.
- *Filter pushdown*: use the min/max statistics to skip row groups that cannot match `WHERE id = 42`.

Partitioning splits data into folders by a column (`dataverse_id=3/`, `extract_date=2026-10-13/`). A filter on the partition column lets the engine skip whole folders without opening them. Choose columns that queries filter on and that have few distinct values (date, region), not ids.

Other principles:

- Parsing JSON or CSV is expensive; do it ONCE, store typed Parquet, and let later stages read that.
- `SELECT *` defeats projection pushdown. Name the columns you need.
- The biggest wins come from reading less data, then from joining smaller inputs, then from tuning settings.
- Look at the plan with `EXPLAIN` (and `EXPLAIN ANALYZE` for timings and row counts) before guessing.
- Indexes matter less in analytic engines: scans over columnar data are the default path.

2. Real-world case

The sample extraction stores 370 dataset records as one 0.5 MB gzipped JSONL file per dataverse. That is tiny today, but `python silver.py` defaults to `--runs all`, so every build re-reads and re-parses EVERY raw run ever made: the cost grows with each new run. Silver exists precisely to pay the parsing cost once and publish typed, compressed Parquet files that gold and dashboards can query quickly. `--runs latest` is the lever when a full rebuild gets slow.

3. Popular usage

- Data lakes and lakehouses: Parquet (or Iceberg / Delta on top) partitioned by date.
- Converting CSV / JSON exports to Parquet as the first step of any analytics pipeline.

- Reading only the needed columns and date range from huge files with DuckDB, Spark, Athena or BigQuery.

4. Example code

Python

```python
# Build 300,000 rows and write them as CSV, JSON lines and Parquet. Compare the sizes.
import os, pathlib, tempfile
tmp = tempfile.TemporaryDirectory()          # removed automatically when the tutor exits
root = pathlib.Path(tmp.name)
con.execute("""
    CREATE TABLE big AS
    SELECT i AS id, 'DS' || (i % 5000) AS global_id, (i % 7) AS dataverse_id,
           DATE '2025-01-01' + CAST(i % 365 AS INTEGER) AS day, repeat('x', 40) AS note
    FROM range(300000) t(i)""")
paths = {name: (root / f"big.{name}").as_posix() for name in ("csv", "json", "parquet")}
con.execute(f"COPY big TO '{paths['csv']}' (FORMAT csv, HEADER)")
con.execute(f"COPY big TO '{paths['json']}' (FORMAT json)")
con.execute(f"COPY big TO '{paths['parquet']}' (FORMAT parquet, COMPRESSION zstd)")
for name, p in paths.items():
    print(f"{name:8} {os.path.getsize(p) / 1024:9,.0f} KB")
con.execute(f"CREATE VIEW big_parquet AS SELECT * FROM read_parquet('{paths['parquet']}')")
con.execute(f"CREATE VIEW big_parquet_meta AS SELECT * FROM parquet_metadata('{paths['parquet']}')")
```

Result

```
csv         19,748 KB
json        35,276 KB
parquet        420 KB
```

SQL

```sql
-- Projection pushdown in action: this query needs 1 of 5 columns, so only that column is read from the Parquet file.
SELECT dataverse_id, count(*) AS n FROM big_parquet GROUP BY dataverse_id ORDER BY dataverse_id;
```

Result

```
dataverse_id | n
-------------+------
0            | 42858
1            | 42857
2            | 42857
3            | 42857
4            | 42857
5            | 42857
6            | 42857
(7 rows)
```

SQL

```sql
-- Statistics for the id column: min/max per row group. A filter like id = 42 can skip every group whose range excludes 42.
SELECT row_group_id, stats_min, stats_max, num_values
FROM big_parquet_meta
WHERE path_in_schema = 'id'
ORDER BY row_group_id;
```

Result

```
row_group_id | stats_min | stats_max | num_values
-------------+-----------+-----------+-----------
0            | 0         | 122879    | 122880
1            | 122880    | 245759    | 122880
2            | 245760    | 299999    | 54240
(3 rows)
```

Python

```python
# Filter pushdown: the WHERE condition appears INSIDE the Parquet scan node of the plan, not as a separate step afterwards.
plan = "\n".join(str(row[1]) for row in con.execute("EXPLAIN SELECT * FROM big_parquet WHERE id = 42").fetchall())
print("filter is part of the Parquet scan:", "id=42" in plan.replace(" ", ""))
```

Result

```
filter is part of the Parquet scan: True
```

Python

```python
# Partitioning: one folder per dataverse_id. A filter on the partition column only needs to open ONE folder.
by_dv = root / "by_dataverse"
con.execute(f"COPY big TO '{by_dv.as_posix()}' (FORMAT parquet, PARTITION_BY (dataverse_id))")
print("folders:", sorted(p.name for p in by_dv.iterdir()))
n = con.sql(f"""SELECT count(*) FROM read_parquet('{by_dv.as_posix()}/*/*.parquet', hive_partitioning = true)
                WHERE dataverse_id = 3""").fetchone()[0]
print("rows with dataverse_id = 3:", n)
```

Result

```
folders: ['dataverse_id=0', 'dataverse_id=1', 'dataverse_id=2', 'dataverse_id=3', 'dataverse_id=4', 'dataverse_id=5', 'dataverse_id=6']
rows with dataverse_id = 3: 42857
```

SQL

```sql
-- Engine settings exist but come last: first read less data. Example: cap the worker threads.
SET threads = 2;
SELECT current_setting('threads') AS threads;
```

Result

```
threads
-------
2
(1 row)
```

LESSON 24

Running SQL safely from Python: parameters, quoting, templates

Level: intermediate

In this project: `_sql_string()`, the `{{RAW_FILES}}` substitution in `build_silver_tables`, `SET s3_...` in `connect_duckdb`

Needs: nothing

1. Theory

SQL is text. If you build that text by pasting untrusted values into it, the value can change the meaning of the statement: **SQL injection**. The defence is to keep code and data separate.

1. **Parameters** (prepared statements): `execute("... WHERE title = ?", [value])` or named `$name` with a dict. The database receives the value separately and never parses it as SQL. Use them for every VALUE in `WHERE`, `INSERT`, `VALUES`.
2. **Quoting literals**: some places cannot take parameters (`SET option = ...`, `COPY ... TO 'path'`, file paths in `read_json`). There, escape the value for SQL: wrap in single quotes and double any single quote inside (`O'Brien` -> `'O''Brien'`).
3. **Identifiers** (table and column names) cannot be parameters either. Take them from a fixed list (whitelist), or quote them with double quotes and double any double quote inside. Never paste user text as an identifier.
4. **Templates**: replacing a placeholder in a SQL file is fine when the replacement is built from trusted, quoted pieces, and when you check that the placeholder was found exactly the expected number of times.
5. **Secrets**: credentials end up in SQL strings (`SET s3_secret_access_key = ...`). Do not log those statements. DuckDB's `CREATE SECRET` keeps them out of ad-hoc statements.

`executemany(sql, rows)` runs one parameterized statement for many rows; it is also much faster than a Python loop of formatted strings.

2. Real-world case

`silver.py` must give DuckDB a list of raw file paths taken from the run manifests, and values for the S3 settings. Those cannot be bound as parameters, so `_sql_string()` quotes them, the table names come from a constant list (`SILVER_TABLES`), and `build_silver_tables` refuses to run unless `{{RAW_FILES}}` appears exactly once outside comments. A dataverse alias or path containing an apostrophe would otherwise break the statement, or in the worst case change it.

3. Popular usage

- Every web application and ETL script that filters on user input.
- ORMs and drivers always use parameters under the hood; hand-written f-string SQL is the usual source of vulnerabilities.
- Parameterized dashboards: the BI tool binds the filter values.

4. Example code

Python

```
# 1) The classic injection. The user types a value that turns the WHERE into "always true".
con.execute("CREATE TABLE datasets AS SELECT * FROM (VALUES ('Rice yield survey'), ('Fish landing records'), ('Soil moisture sensors')) AS
t(title)")
user_input = "x' OR '1'='1"

unsafe_sql = f"SELECT title FROM datasets WHERE title = '{user_input}'"
print("statement  :", unsafe_sql)
print("f-string   :", con.sql(unsafe_sql).fetchall())                                  # every row leaks
print("parameter  :", con.execute("SELECT title FROM datasets WHERE title = ?", [user_input]).fetchall())  # no match
```

Result

```
statement  : SELECT title FROM datasets WHERE title = 'x' OR '1'='1'
f-string   : [('Rice yield survey',), ('Fish landing records',), ('Soil moisture sensors',)]
parameter  : []
```

Python

```
# 2) Named parameters and executemany.
print(con.execute("SELECT $a + $b AS total", {"a": 40, "b": 2}).fetchall())
con.execute("CREATE TABLE tags (global_id VARCHAR, tag VARCHAR)")
con.executemany("INSERT INTO tags VALUES (?, ?)", [("DS1", "rice"), ("DS1", "yield"), ("DS3", "fish")])
print(con.sql("SELECT global_id, count(*) AS n FROM tags GROUP BY global_id ORDER BY 1").fetchall())
```

Result

```
[(42,)]
[('DS1', 2), ('DS3', 1)]
```

Python

```
# 3) Where parameters are impossible (paths, SET values): quote the literal, as silver._sql_string does.
def sql_string(value):
    return "'" + str(value).replace("'", "''") + "'"

path = "s3://raw/dataverse_search/O'Brien/part-00000.jsonl.gz"
print("quoted literal:", sql_string(path))
print("round trip    :", con.sql(f"SELECT {sql_string(path)} AS p").fetchone()[0] == path)
```

Result

```
quoted literal: 's3://raw/dataverse_search/O''Brien/part-00000.jsonl.gz'
round trip    : True
```

Python

```
# 4) Identifiers: whitelist them. Never paste user text where a table name goes.
ALLOWED_TABLES = {"datasets", "tags"}

def count_rows(table):
    if table not in ALLOWED_TABLES:
        raise ValueError(f"table not allowed: {table!r}")
    return con.sql(f'SELECT count(*) FROM "{table}"').fetchone()[0]

print("datasets:", count_rows("datasets"))
try:
    count_rows("datasets; DROP TABLE datasets")
except ValueError as e:
    print("rejected:", e)
```

Result

```
datasets: 3
rejected: table not allowed: 'datasets; DROP TABLE datasets'
```

Python

```python
# 5) A checked template, like build_silver_tables: replace the placeholder only on code lines, and insist on exactly one.
def fill_template(template, files):
    file_list = "[" + ", ".join(sql_string(f) for f in files) + "]"
    lines, replaced = [], 0
    for line in template.splitlines():
        if not line.lstrip().startswith("--") and "{{RAW_FILES}}" in line:
            line = line.replace("{{RAW_FILES}}", file_list)
            replaced += 1
        lines.append(line)
    if replaced != 1:
        raise ValueError(f"placeholder must appear exactly once outside comments (found {replaced})")
    return "\n".join(lines)

template = "-- reads {{RAW_FILES}} (comment: left alone)\nSELECT * FROM read_json({{RAW_FILES}}, format = 'newline_delimited')"
print(fill_template(template, ["a.jsonl", "b'c.jsonl"]))
try:
    fill_template("SELECT 1 -- no placeholder here", [])
except ValueError as e:
    print("rejected:", e)
```

Result

```
-- reads {{RAW_FILES}} (comment: left alone)
SELECT * FROM read_json(['a.jsonl', 'b''c.jsonl'], format = 'newline_delimited')
rejected: placeholder must appear exactly once outside comments (found 0)
```

LESSON 25

Dialect portability: DuckDB versus PostgreSQL (and the SQL standard)

Level: advanced | In this project: `QUALIFY`, `ANTI JOIN`, `GROUP BY ALL`, `TRY_CAST`, JSONPath `->>`, `CREATE OR REPLACE TABLE` | Needs: lessons 09, 12, 21

1. Theory

SQL is a standard with many dialects. The core (`SELECT`, joins, `GROUP BY`, CTEs, window functions, `UNION`, `CASE`, `COALESCE`) is portable. Differences appear in: convenience syntax, JSON and array handling, date functions, DDL details, and upsert syntax. Engines also differ in NULL ordering defaults and integer division.

Where you meet this: the project computes in DuckDB, and its `config.py` already lists PostgreSQL settings for a later stage.

DuckDB (used here)	PostgreSQL	Portable choice
`QUALIFY row_number() OVER (...) = 1`	subquery with `WHERE rn = 1`, or `DISTINCT ON (key)`	subquery / CTE with `rn`
`a ANTI JOIN b USING (k)`	`NOT EXISTS (SELECT 1 FROM b WHERE ...)`	`NOT EXISTS`
`TRY_CAST(x AS DATE)`	no `TRY_CAST` (PostgreSQL 16+ has `pg_input_is_valid()` to test a cast first)	`CASE WHEN <valid pattern> THEN CAST(...) END`
`value ->> '$.a.b'` (JSONPath)	`jsonb ->> 'a'`, `#>> '{a,b}'`, `jsonb_path_query`	keep JSON parsing in the engine that owns it
`GROUP BY ALL`, `SELECT * EXCLUDE (c)`, `FROM t`	not available	list the columns explicitly
`list(x)` / `array_agg(x)`	`array_agg(x)`	`array_agg` (DuckDB accepts it)
`string_agg(x, ',' ORDER BY x)`	same	already portable
`x FILTER (WHERE ...)`	same	already portable (not in MySQL / SQL Server: use `CASE`)
`UNNEST(l) WITH ORDINALITY`	`unnest(arr) WITH ORDINALITY` (in a lateral `FROM`)	similar
`CREATE OR REPLACE TABLE t AS ...`	`DROP TABLE IF EXISTS t; CREATE TABLE t AS ...`	drop + create in a transaction
`strftime(ts, '%Y-%m')`	`to_char(ts, 'YYYY-MM')`	`date_trunc('month', ts)`
`7 / 2` is `3.5`	integer `7 / 2` is `3`	`CAST(7 AS DECIMAL) / 2`

Strategies:

1. **Use the engine's strengths where they live.** Heavy flattening and parsing in DuckDB; tables for applications and upserts in PostgreSQL. Move typed data between them (Parquet, `COPY`), not DuckDB-specific SQL.
2. **Prefer standard forms when it costs nothing** (`NOT EXISTS`, CTEs, `CASE`, `array_agg`).
3. **Test on the target engine.** A query that parses in one engine can behave differently (NULL order, rounding).
4. **Tools**: `sqlglot` can translate between dialects, dbt adapters hide many differences, and ANSI-compliant subsets keep options open. Always review translated SQL.

2. Real-world case

The silver dedup uses `QUALIFY`, which PostgreSQL does not have. When the gold tables move to PostgreSQL, the same logic must be rewritten as a subquery with `row_number()` (or `DISTINCT ON`). The orphan check uses `ANTI JOIN`, which becomes `NOT EXISTS`. Because both forms return the same rows, the project can switch without changing results, as long as the rewrite is tested. The portable forms below prove the equivalence on real sample data.

3. Popular usage

- Migrating a warehouse (Redshift to Snowflake, SQL Server to PostgreSQL) and rewriting dialect-specific SQL.
- Libraries and BI tools that generate SQL for many engines (they stick to the portable core).
- Prototyping in DuckDB (fast, local) and deploying the final logic on a production database.

4. Example code

Python

```python
run_sql_file("setup_bronze.sql")
```

SQL

```sql
-- Sample tables for the comparisons.
CREATE TEMP TABLE versions AS
SELECT data.identifier AS id, run_id, TRY_CAST(data.latestVersion.lastUpdateTime AS TIMESTAMPTZ) AS updated
FROM bronze_meta;
CREATE TEMP TABLE all_ids AS SELECT * FROM (VALUES ('DS1'), ('DS2'), ('DS3'), ('DS4'), ('DS9')) AS t(id);
SELECT count(*) AS version_rows FROM versions;
```

Result

```
version_rows
------------
6
(1 row)
```

Python

```python
# DuckDB's QUALIFY versus the portable subquery: they must return exactly the same rows.
duck = con.sql("""
    SELECT id, run_id FROM versions
    QUALIFY row_number() OVER (PARTITION BY id ORDER BY updated DESC NULLS LAST, run_id DESC) = 1
    ORDER BY id""").fetchall()
portable = con.sql("""
    SELECT id, run_id
    FROM (SELECT *, row_number() OVER (PARTITION BY id ORDER BY updated DESC NULLS LAST, run_id DESC) AS rn FROM versions)
    WHERE rn = 1
    ORDER BY id""").fetchall()
print("QUALIFY == subquery:", duck == portable, "| rows:", len(duck))
```

Result

```
QUALIFY == subquery: True | rows: 4
```

Python

```python
# ANTI JOIN versus NOT EXISTS: ids that never appear in any run (here only DS9).
duck = con.sql("SELECT id FROM all_ids ANTI JOIN versions USING (id) ORDER BY id").fetchall()
portable = con.sql("""SELECT id FROM all_ids a
                      WHERE NOT EXISTS (SELECT 1 FROM versions v WHERE v.id = a.id) ORDER BY id""").fetchall()
print("ANTI JOIN == NOT EXISTS:", duck == portable, "| result:", duck)
```

Result

```
ANTI JOIN == NOT EXISTS: True | result: [('DS9',)]
```

SQL

```sql
-- TRY_CAST has no PostgreSQL twin. A portable pattern: validate with a pattern first, then CAST only valid text.
-- (DuckDB: regexp_matches; PostgreSQL: the ~ operator. The CASE shape is the same.)
SELECT x,
       CASE WHEN regexp_matches(x, '^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$') THEN CAST(x AS DATE) END AS safe_date
FROM (VALUES ('2025-03-01'), ('2025-13-45'), ('not a date')) AS t(x);
```

Result

```
x          | safe_date
-----------+-----------
2025-03-01 | 2025-03-01
2025-13-45 | NULL
not a date | NULL
(3 rows)
```

SQL

```sql
-- DuckDB "friendly SQL" shortcuts: handy while exploring, not portable. The explicit form works everywhere.
SELECT id, count(*) AS n FROM versions GROUP BY ALL ORDER BY id;
```

Result

```
id  | n
----+--
DS1 | 2
DS2 | 2
DS3 | 1
DS4 | 1
(4 rows)
```

SQL

```sql
-- The portable equivalent of the line above, and a portable array aggregate (array_agg) instead of DuckDB's list().
SELECT id, count(*) AS n, array_agg(run_id ORDER BY run_id) AS runs FROM versions GROUP BY id ORDER BY id;
```

Result

```
id  | n | runs
----+---+--------------------------------------------------
DS1 | 2 | [20261012T000000Z_aaaaaa, 20261013T000000Z_bbbbbb]
DS2 | 2 | [20261012T000000Z_aaaaaa, 20261013T000000Z_bbbbbb]
DS3 | 1 | [20261013T000000Z_bbbbbb]
DS4 | 1 | [20261012T000000Z_aaaaaa]
(4 rows)
```

PostgreSQL (shown, not run)

```postgresql
-- PostgreSQL versions of the DuckDB-specific pieces used in this project. (Not run here.)

-- latest row per key (instead of QUALIFY)
SELECT DISTINCT ON (id) id, run_id, updated FROM versions ORDER BY id, updated DESC NULLS LAST, run_id DESC;

-- JSON text extraction from a jsonb column (instead of value ->> '$.authorName.value')
SELECT value #>> '{0,authorName,value}' AS first_author FROM stg_fields WHERE type_name = 'author';

-- unnest with position, in a lateral FROM
SELECT f.global_id, a.pos, a.item -> 'authorName' ->> 'value' AS author_name FROM stg_fields f
CROSS JOIN LATERAL jsonb_array_elements(f.value) WITH ORDINALITY AS a(item, pos);

-- replace a table inside one transaction (instead of CREATE OR REPLACE TABLE)
BEGIN;
DROP TABLE IF EXISTS gold.datasets;
CREATE TABLE gold.datasets AS SELECT * FROM staging.datasets;
COMMIT;
```